

**Міністерство освіти і науки України
Чернівецький національний університет
імені Юрія Федьковича**

кафедра комп'ютерних наук
(назва кафедри)

ЗВІТ

з переддипломної практики
(вид практики)

на базі: Асоціація «Кластер Буковинських інноваційних технологій
імені Йозефа Шумпетера»
(назва бази практики)

Студента групи 444А
(номер групи)

Мулик Андрій
(прізвище, ім'я, по батькові)

Початок практики «04» травня 2026 р.

Закінчення практики «31» травня 2026 р.

Керівники практики:

Від бази практики _____ директор, Юрій УШЕНКО
(підпис) (посада, ім'я, прізвище)

Від кафедри _____ асистент, Віктор ІВАШКО
(підпис) (посада, ім'я, прізвище)

Звіт захищено з оцінкою: _____

« ____ » _____ 20 ____ р.

м. Чернівці
2026 р.

РЕФЕРАТ

У рамках дипломної роботи виконано етапи дослідження, аналізу, проєктування та розробки самовідновлювального конвеєру обробки текстових відгуків на основі Apache Airflow, локальної великої мовної моделі Llama3.2 та колонкового аналітичного сховища DuckDB. Система реалізує повний цикл обробки даних із вбудованою логікою автоматичного виявлення та усунення дефектів вхідних даних і інтегрованим аналізом тональності.

У теоретичній частині проведено аналіз проблеми якості даних у краудсорсингових датасетах на прикладі Yelp Academic Dataset, розглянуто концепцію self-healing у конвеєрах обробки даних та досліджено еволюцію методів аналізу тональності — від лексичних словників до трансформерних LLM. Виконано порівняльний огляд оркестраторів конвеєрів (Apache Airflow, Prefect, Dagster) та обґрунтовано вибір технологічного стеку.

Практична частина включає реалізацію багатокомпонентної системи: двох DAG-ів Apache Airflow із шестикроковою структурою задач (TaskFlow API), детерміністичного модуля самовідновлення на основі патерну «Ланцюжок відповідальності» з п'ятьма класами дефектів, модуля аналізу тональності через Ollama REST API, рівня зберігання на базі DuckDB і SQLite, механізму автентифікації через Bearer-токени та веб-дашборду на базі Blazor Hosted WebAssembly. Конвеєр генерує структуровані health-звіти зі статусами HEALTHY / WARNING / DEGRADED / CRITICAL та підтримує принцип graceful degradation при відмові зовнішніх сервісів.

Ключові слова: самовідновлення, ETL, конвеєр обробки даних, аналіз тональності, Apache Airflow, великі мовні моделі, Ollama, Llama3.2, DuckDB, Blazor, graceful degradation, Yelp Academic Dataset.

Робота містить результати власних досліджень. Використання ідей, результатів і текстів наукових досліджень інших авторів супроводжується посиланням на відповідне джерело.

ЗМІСТ

РЕФЕРАТ	2
ВСТУП	5
РОЗДІЛ 1	9
ТЕОРЕТИЧНІ ОСНОВИ ТА ОГЛЯД ПРЕДМЕТНОЇ ГАЛУЗІ	9
1.1. Проблема якості даних у реальних датасетах (на прикладі Yelp Academic Dataset)	9
1.2. Концепція Self-Healing у конвеєрах обробки даних.....	12
1.3. Sentiment Analysis: підходи та моделі.....	14
1.4. Огляд технологій: Apache Airflow, Ollama, Blazor, llama3.2	17
1.5. Постановка задачі.....	20
РОЗДІЛ 2	23
ПРОЄКТУВАННЯ СИСТЕМИ.....	23
2.1. Загальна архітектура системи	23
2.2. Проєктування рівня зберігання даних (DuckDB)	25
2.3. Проєктування конвеєрів Apache Airflow	26
2.4. Проєктування модуля самовідновлення	27
2.5. Проєктування модуля аналізу тональності	29
2.6. Проєктування структури вихідних даних та health-звіту	30
2.7. Проєктування Blazor-дашборду.....	31
РОЗДІЛ 3	36
РЕАЛІЗАЦІЯ СИСТЕМИ.....	36
3.1. Технологічний стек та середовище розробки	36
3.2. Реалізація рівня зберігання даних	37
3.3. Реалізація конвеєрів Apache Airflow	38
3.4. Реалізація модуля самовідновлення.....	39
3.5. Реалізація модуля аналізу тональності	40
3.6. Реалізація автентифікації та авторизації	40
3.7. Реалізація Blazor-дашборду	42
3.8. Розгортання системи.....	45
РОЗДІЛ 4	47
ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ	47

4.1. Методологія тестування	47
4.2. Тестування модуля самовідновлення.....	48
4.3. Тестування конвеєру Apache Airflow.....	49
4.4. Тестування Blazor-дашборду	51
4.5. Аналіз результатів sentiment analysis	54
4.6. Аналіз продуктивності системи.....	56
4.7. Порівняння з аналогами	57
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	63
ДОДАТОК А.....	67
Реалізація рівня зберігання даних (DuckDB)	67
ДОДАТОК Б	72
Реалізація конвеєрів Apache Airflow (Python).....	72
ДОДАТОК В.....	75
Реалізація Self-Healing та LLM-аналізу (Python)	75
ДОДАТОК Г	80
Реалізація автентифікації та авторизації (C#)	80
ДОДАТОК Д.....	86
AirflowService, Polling, Program.cs, SVG (C#).....	86
ДОДАТОК Е	92
Конфігурація розгортання.....	92

ВСТУП

Актуальність теми

Обробка даних стала критичною інфраструктурою для організацій у будь-якій галузі. За оцінками DataOps Impact Assessment 2023, підприємства обробляють у середньому 2,7 петабайти даних на місяць через власну конвеєрну інфраструктуру, витрачаючи при цьому близько 41% часу інженерів даних на усунення проблем якості. У такому контексті надійність та відмовостійкість конвеєрів обробки даних набуває не меншого значення, ніж їх функціональність.

Аналіз тональності тексту (Sentiment Analysis) залишається одним із найбільш масово затребуваних застосувань обробки природної мови у виробничих системах: від рекомендаційних сервісів до систем моніторингу репутації бренду. Yelp Academic Dataset — один із найпоширеніших публічних датасетів у цій галузі — містить мільйони реальних відгуків і водночас є показовим прикладом краудсорсингових даних із характерними дефектами якості: відсутніми значеннями, хибними типами, порожніми текстами та надмірно довгими записами.

Паралельно спостерігається стрімке зростання застосування великих мовних моделей (Large Language Models, LLM) для задач класифікації тексту. Поява відкритих моделей — зокрема сімейства LLaMA від Meta AI — та інструментів локального інференсу, таких як Ollama, відкрила можливість розгортання LLM безпосередньо на власному обладнанні без передачі даних до хмарних провайдерів. Водночас Apache Airflow утвердився як індустріальний стандарт оркестрації конвеєрів обробки даних, охоплюючи 55% організацій, що використовують його щоденно.

Однак в україномовній академічній літературі відсутні комплексні дослідження, які системно охоплюють: (1) теоретичні основи концепції self-healing у конвеєрах обробки даних; (2) практичну реалізацію автоматизованого виявлення та виправлення дефектів якості даних; (3)

інтеграцію локальних LLM у виробничі конвеєри; (4) побудову веб-орієнтованих інструментів моніторингу таких систем.

Основна задача дослідження — розробка самовідновлювального конвеєру аналізу тональності текстових відгуків із вбудованою логікою автоматичного виявлення та усунення дефектів вхідних даних на основі Apache Airflow, локальної LLM llama3.2 через Ollama та аналітичного сховища DuckDB.

Конкретні завдання:

1. Провести теоретичне дослідження концепції self-healing у контексті конвеєрів обробки даних та огляд методів аналізу тональності тексту.
2. Проаналізувати проблему якості даних у реальних краудсорсингових датасетах на прикладі Yelp Academic Dataset.
3. Розробити архітектуру багатокомпонентної системи з рівнями обробки, зберігання та відображення.
4. Реалізувати детерміністичний алгоритм самовідновлення з п'ятьма класами дефектів та відповідними стратегіями виправлення.
5. Інтегрувати локальну LLM llama3.2 через Ollama для аналізу тональності без передачі даних зовнішнім провайдерам.
6. Побудувати веб-дашборд на базі Blazor для моніторингу та аудиту запусків конвеєру.

Мета роботи — продемонструвати комплексний підхід до інтеграції механізмів самовідновлення, локального LLM-інференсу та веб-орієнтованого моніторингу в єдиній виробничій системі, що може слугувати зразком для аналогічних рішень у галузі DataOps та MLOps.

Конкретні цілі:

- синтез теоретичних знань про self-healing архітектури та методи аналізу тональності;
- дослідження проблематики якості даних у реальних датасетах та систематизація типів дефектів;

- розробка практичної методології побудови відмовостійких конвеєрів із врахуванням принципу graceful degradation;
- демонстрація принципів локального LLM-інференсу через Ollama та llama3.2;
- формулювання практичних рекомендацій щодо вибору технологічного стеку для DataOps-систем.

Об'єктом дослідження є конвеєр обробки та аналізу текстових даних у виробничих системах з урахуванням вимог до якості та відмовостійкості. Конкретно:

- механізми автоматичного виявлення та класифікації дефектів вхідних даних;
- архітектура оркестрації задач на базі Apache Airflow;
- інтеграція локальних LLM у задачах класифікації тексту;
- підсистема аналітики та моніторингу на базі DuckDB і Blazor.

Предметом дослідження є принципи, методики та технічні засоби проектування і реалізації самовідновлювального конвеєру аналізу тональності:

1. Концептуальні засади self-healing та його застосування в ETL-конвеєрах;
2. Алгоритмічні рішення при побудові детерміністичної логіки відновлення;
3. Деталізовані процедури інтеграції локального LLM-інференсу через Ollama REST API;
4. Практичне застосування DuckDB як вбудованого аналітичного сховища;
5. Аналітичні аспекти: оцінка якості аналізу тональності, продуктивності системи та порівняння підходів.

Дослідження спирається на:

- системний підхід до розглядання конвеєру як цілісної системи з взаємопов'язаними рівнями;
- архітектурний аналіз компонентів та їх взаємовідносин;
- практичний емпіризм з документуванням реального case study на Yelp Academic Dataset;
- порівняльний аналіз методів оркестрації (Airflow, Prefect, Dagster) та підходів до аналізу тональності;
- синтез індустріальних best practices у галузі DataOps та MLOps.

Для комплексного розв'язання окресленої задачі необхідно спочатку сформулювати міцний теоретичний фундамент. У першому розділі роботи здійснюється детальний огляд проблеми якості даних у реальних датасетах, концепції self-healing у конвеєрах обробки даних, еволюції методів аналізу тональності — від лексичних словників до трансформерних LLM — та ключових технологій системи: Apache Airflow, Ollama, llama3.2 і Blazor. Розуміння цих фундаментальних концепцій створює необхідну теоретичну базу для осмисленого проектування архітектури та реалізації системи, що описуються в наступних розділах.

РОЗДІЛ 1

ТЕОРЕТИЧНІ ОСНОВИ ТА ОГЛЯД ПРЕДМЕТНОЇ ГАЛУЗІ

1.1. Проблема якості даних у реальних датасетах (на прикладі Yelp Academic Dataset)

У сучасному середовищі обробки даних якість вхідних даних є одним із ключових чинників, що визначають надійність і цінність будь-якого аналітичного конвеєра. Реальні набори даних, зібрані з відкритих джерел, неминуче містять помилки, пропуски та структурні невідповідності, що суттєво ускладнює їх автоматизовану обробку [1].

Yelp Academic Dataset є одним із найпоширеніших публічних наборів даних у галузі аналізу природної мови та рекомендаційних систем. Він містить мільйони відгуків реальних користувачів про підприємства і бізнес-об'єкти і широко використовується в академічних дослідженнях [2]. Разом із тим, подібно до інших краудсорсингових датасетів, він страждає на численні проблеми з якістю даних, які необхідно враховувати перед будь-яким аналізом.

Основні типи аномалій у реальних текстових датасетах можна систематизувати за декількома категоріями. Відсутні значення (NULL або порожні поля) виникають у ситуаціях, коли користувач залишив відгук без текстового коментаря. Хибний тип даних трапляється, коли числовий або булевий параметр потрапляє в текстове поле через програмні помилки або нестандартне заповнення форми. Порожній або беззмістовний текст відгуків (пробіли, лише спецсимволи) становить окрему категорію шуму, що не несе аналітичної цінності. Надмірно довгі записи, що перевищують технічні ліміти моделей обробки природної мови, можуть призводити до помилок виконання або неточних результатів аналізу [3].

Дослідження Аудиту рекомендаційної системи Yelp [3] демонструє, що платформа Yelp самостійно фільтрує частину відгуків, переміщуючи їх

до категорії «не рекомендованих» (Not Recommended) через конфлікт інтересів, недостатню надійність або надуманість. Ці фільтровані відгуки не видаляються з платформи, але приховуються від загального перегляду, що спричиняє додаткові труднощі при побудові навчальних датасетів.

Поздовжнє дослідження відгуків Yelp [4], у якому було відстежено більш ніж 2 мільйони відгуків понад 11 тисяч підприємств, виявило феномен «reclassification» — повторної класифікації відгуків з часом. Показники перекласифікації варіюються від 0,54% за чотири місяці до 8,69% за вісім років, що свідчить про динамічну та нестабільну природу датасету.

Таблиця 1.1 – Типи помилок у даних та стратегії їх виправлення у системі
Self-Healing Pipeline

Тип помилки	Опис	Стратегія виправлення	Код статусу
missing_text	Значення поля "text" відсутнє (None/null)	Заповнення placeholder-текстом "No review text provided."	healed
wrong_type	Поле "text" має не рядковий тип (int, float, bool)	Примусове перетворення типу (str())	healed
empty_text	Текст містить лише пробіли	Заповнення placeholder-текстом	healed
special_characters_only	Текст не містить жодного алфавітно-цифрового символу	Заміна на "[Non-text content]"	healed
too_long	Довжина тексту перевищує MAX_TEXT_LENGTH (2000 символів)	Обрізка до 2000 символів з "..."	healed
—	Текст коректний	Нормалізація пробілів (strip())	success

Важливою характеристикою Yelp Academic Dataset є нерівномірний розподіл зіркових рейтингів: відгуки з оцінкою 1 та 5 зірок становлять непропорційно велику частку, що ускладнює задачі аналізу тональності тексту [5]. Більш того, як зазначають Ткачук та Ю у дослідженні ресторанних

відгуків [6], моделі аналізу тональності, навчені на несбалансованих даних, демонструють виражену тенденцію до помилкової класифікації відгуків зі складним або змішаним емоційним навантаженням.

Проблема якості даних набуває особливого значення в контексті розробки промислових конвеєрів обробки даних. За оцінкою DataOps Impact Assessment 2023 [7], підприємства обробляють у середньому 2,7 петабайти даних на місяць через свою конвеєрну інфраструктуру, при цьому інженери даних витрачають близько 41% свого робочого часу на усунення проблем якості. Ця статистика підкреслює нагальну потребу у впровадженні механізмів автоматичного виявлення і виправлення помилок у потоках обробки даних.

Розподіл помилок у відгуках

ДАНІ

ДЖЕРЕЛО

Review Distribution

Yelp / Тексти відгуків

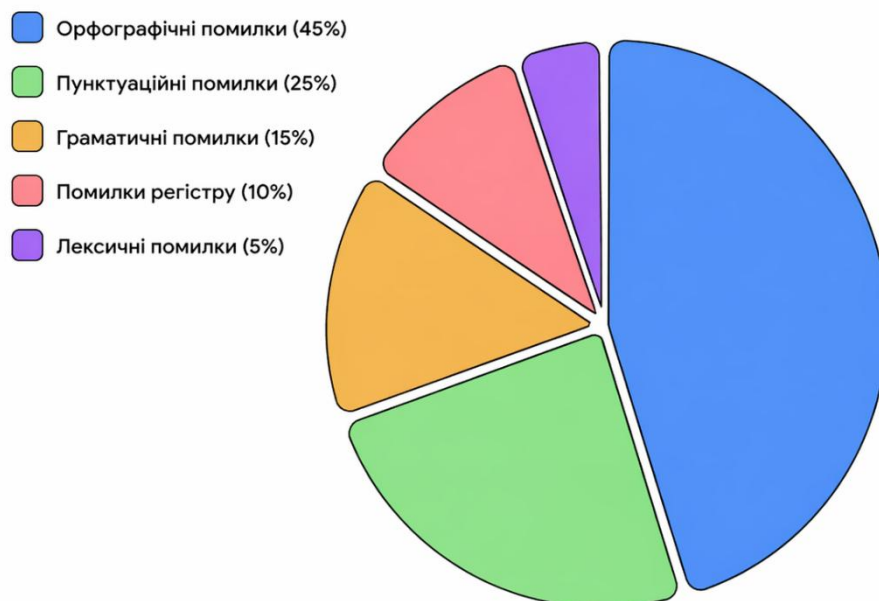


Рисунок 1.1 – Типовий розподіл типів аномалій у краудсорсингових датасетах відгуків

Таким чином, розробка систем з вбудованою логікою самовідновлення (self-healing) є актуальним науково-практичним завданням, що вирішує реальну проблему деградації якості даних у виробничих системах.

1.2. Концепція Self-Healing у конвеєрах обробки даних

Концепція самовідновлення (self-healing) у програмних системах передбачає здатність системи автоматично виявляти, діагностувати та усувати збої без зовнішнього втручання. Вперше розвинута у контексті автономних обчислювальних систем, вона знайшла широке застосування у хмарних та розподілених архітектурах, ETL-конвеєрах та мережесистемах [8].

У контексті конвеєрів обробки даних self-healing означає автоматичне відновлення після збоїв, що можуть виникати на будь-якому з етапів: завантаження, трансформація, збереження або передача даних. Традиційні конвеєри є жорсткими, непрозорими системами, що відмовляють у тиші — без видимих повідомлень про помилки, що спричиняє каскадні збої і порушення цілісності даних [9].

Згідно з дослідженням Shah та Hazarika [10] щодо самовідновлювальних ETL-конвеєрів, ключовими компонентами такої архітектури є: (1) рівень безперервного моніторингу, що здійснює спостереження за операціями конвеєра в реальному часі; (2) механізм діагностики та класифікації типів помилок; (3) реєстр стратегій виправлення, що містить готові процедури відновлення для різних сценаріїв збоїв; (4) компонент звітності про стан здоров'я системи.

Дослідження у галузі самовідновлення на основі методів машинного навчання отримало значний імпульс із появою робіт з підкріплення навчання. Робота, представлена на конференції IEEE [11], пропонує конвеєр самовідновлення на основі глибокого Q-навчання (DQN) і проксимальної оптимізації стратегій (PPO), що дозволяє динамічно оптимізувати стратегії

нейтралізації помилок. Хоча цей підхід є потужнішим за прості статичні правила, він вимагає значних обчислювальних ресурсів і великих обсягів навчальних даних.

Практичнішим підходом для більшості виробничих систем є визначальний (детерміністичний) self-healing на основі класифікованих правил. Samuele та співавтори [12] описують архітектуру самовідновлювальних конвеєрів з автономним усуненням помилок, де кожен тип збою відображається на конкретну стратегію відновлення — від простого заповнення відсутніх значень до ескалації до оператора у разі критичного рівня деградації.

Таблиця 1.2 – Порівняльна характеристика підходів до реалізації Self-Healing у конвеєрах даних

Підхід	Принцип роботи	Переваги	Недоліки	Приклади
Правила (rule-based)	Явно задані умови if-else для кожного типу збою	Передбачуваність, простота, висока швидкість	Потребує ручного оновлення правил	Airflow retry policies
ML-based (RL)	Агент навчається оптимальним діям на основі сигналів підкріплення	Адаптивність до нових сценаріїв	Потребує великої кількості даних і часу навчання	DQN/PPO pipelines [11]
Meta-learning (MAML)	Швидка адаптація до нових типів збоїв за малою кількістю прикладів	Висока узагальнювальна здатність	Складна імплементація	Research prototypes
Гібридний	Комбінація правил і статистичних аномалій	Баланс між передбачуваністю та адаптивністю	Складніша архітектура	Даний проєкт

Дослідження Aldrini та ін. [13] підкреслює, що системи самовідновлення для виробництва повинні розрізняти транз'єнтні збої, що вимагають терпимості, і стійкі відмови, що потребують негайного виправлення. Авторами запропоновано концепцію багаторівневих перевірок

здоров'я (multi-level health checking), що охоплює часові масштаби від мілісекунд до годин.

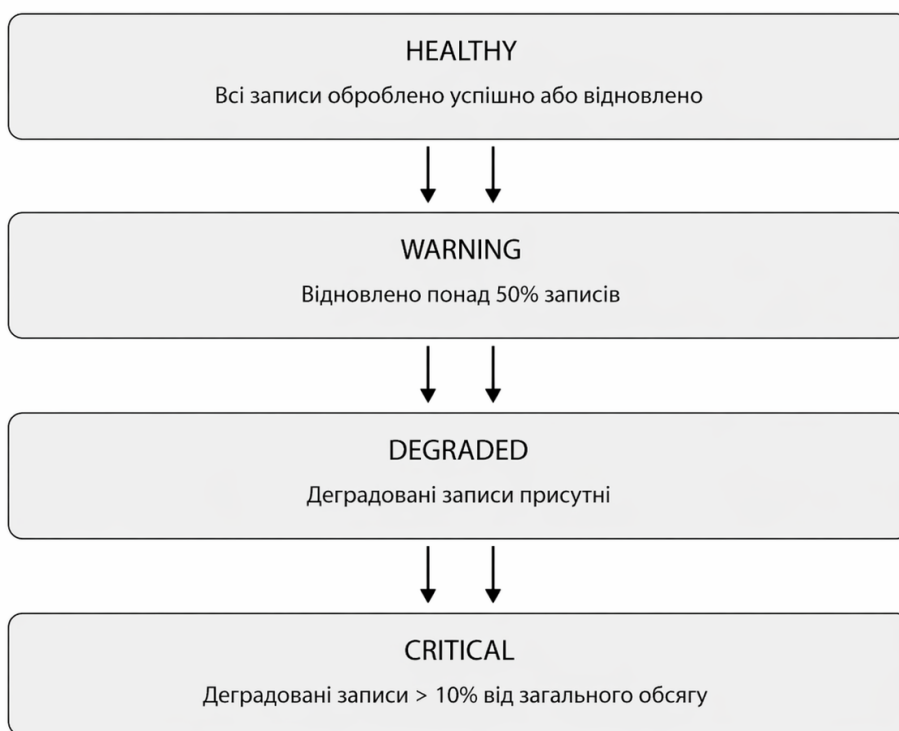


Рисунок 1.2 – Узагальнена схема Self-Healing конвеєра обробки даних

У контексті даної роботи впроваджена система самовідновлення реалізує детерміністичний підхід з чотирма рівнями стану здоров'я конвеєра: **HEALTHY** (всі записи оброблено успішно або відновлено), **WARNING** (відновлено понад 50% записів), **DEGRADED** (є деградовані записи), **CRITICAL** (деградовані записи перевищують 10% від загального обсягу). Такий підхід забезпечує прозорість і аудитабельність роботи системи.

1.3. Sentiment Analysis: підходи та моделі

Аналіз тональності (Sentiment Analysis, SA) — одна з фундаментальних задач обробки природної мови (Natural Language Processing, NLP), що полягає у визначенні емоційного забарвлення тексту: позитивного (POSITIVE), негативного (NEGATIVE) або нейтрального (NEUTRAL).

Задача виникла у 2000-х роках і з того часу пройшла кілька еволюційних стадій, від лексичних словників до трансформерних архітектур [14].

Lexicon-based підходи використовують заздалегідь складені словники емоційних слів (наприклад, SentiWordNet, VADER) та агрегують їх оцінки для визначення полярності тексту. Попри свою простоту та інтерпретованість, такі методи погано справляються з саркастичними, контекстно-залежними або розмовними текстами.

Машинне навчання (Naive Bayes, SVM, Logistic Regression) на основі TF-IDF або bag-of-words представлень суттєво покращило якість SA, проте не здатне враховувати довготривалі семантичні залежності в тексті. Глибокі нейронні мережі (LSTM, CNN) зробили наступний крок, впровадивши механізми уваги, проте вимагали великих розмічених датасетів.

Револьюційним стало впровадження трансформерних архітектур, зокрема BERT (Bidirectional Encoder Representations from Transformers) у 2018 році [15]. BERT та його похідні (RoBERTa, ALBERT, XLNet) досягли рівня людини на стандартних бенчмарках SA завдяки попередньому навчанню на гігантських корпусах тексту і донавчанню на конкретних задачах.

З появою великих мовних моделей (Large Language Models, LLM) — GPT-3, GPT-4, LLaMA — парадигма SA знову зазнала змін. Zhang та ін. [16] провели комплексне дослідження на 26 датасетах і 13 задачах SA, показавши, що LLM демонструють задовільну ефективність у простих задачах класифікації, але поступаються спеціалізованим моделям у складних задачах з глибоким розумінням аспектів тональності. Водночас LLM значно переважають малі спеціалізовані моделі в умовах few-shot навчання.

Порівняльне дослідження LLM та спеціалізованих нейронних мереж для аналізу тональності опитувань [17] демонструє, що моделі на базі BERT (RoBERTa) досягають близько 87% точності на стандартних метриках у класифікації студентських відгуків, тоді як LLM типу GPT-3.5 та Llama-3

показують конкурентні результати без будь-якого тонкого налаштування (zero-shot). Ця здатність до узагальнення є ключовою перевагою LLM у контексті конвеєра, де немає можливості виконати донавчання на конкретному доменному датасеті.

Таблиця 1.3 – Порівняльний аналіз підходів до аналізу тональності тексту

Підхід	Представники	F1-score (avg)	Режим навчання	Застосування
Lexicon-based	VADER, SentiWordNet	0.55–0.70	Без навчання	Швидкий попередній аналіз
Класичне ML	SVM, Naive Bayes	0.70–0.80	Supervised	Збалансовані датасети
Deep Learning	LSTM, CNN, BiLSTM	0.78–0.85	Supervised	Великі датасети
Transformer (fine-tuned)	BERT, RoBERTa	0.87–0.93	Fine-tuning	Доменний аналіз
LLM (zero-shot)	GPT-4, LLaMA-3.2	0.75–0.88	Zero/Few-shot	Гнучкий аналіз без навчання

Порівняння LLM та людських анотаторів у задачах аналізу тональності [18] показало, що великі мовні моделі досягають міжочінювочної надійності (Krippendorff's alpha), що порівнянна з угодою між людьми-анотаторами на задачах бінарної і тривірневої класифікації. Це відкриває можливість для використання LLM у якості повністю автоматичних анотаторів у виробничих конвеєрах.

Хронологія розвитку методів аналізу тональності тексту

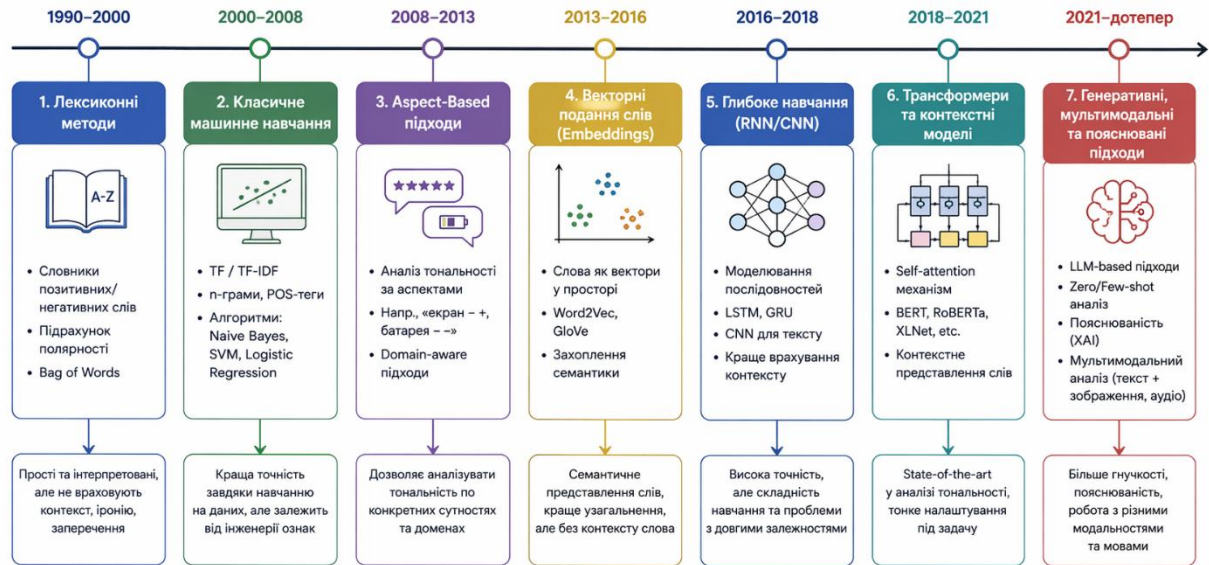


Рисунок 1.3 – Хронологія розвитку методів аналізу тональності тексту

Для задач аналізу тональності відгуків ресторанів [6] з використанням локальних LLM на основі методу голосування більшості (majority voting) показано, що LLaMA 2 перевищує показники CNN і BiLSTM-моделей при аналізі коротких текстів відгуків, демонструючи найвищу точність у категоріях з екстремальними рейтингами (1 або 5 зірок). Це підтверджує доцільність застосування LLM у системі, розробленій у даній роботі.

1.4. Огляд технологій: Apache Airflow, Ollama, Blazor, llama3.2

1.4.1. Apache Airflow

Apache Airflow — це відкрита платформа оркестрації робочих процесів, розроблена в Airbnb у 2014 році і передана під крило Apache Software Foundation у 2019 році [19]. Платформа дозволяє інженерам визначати, планувати та оркеструвати робочі процеси у вигляді спрямованих ациклічних графів (Directed Acyclic Graphs, DAG), реалізованих як Python-код.

Парадигма «Workflows as Code» (WaC) є ключовим принципом Airflow і забезпечує декілька критично важливих переваг над традиційними

підходами: динамічне генерування конвеєрів за допомогою стандартних конструкцій Python, версійний контроль через системи VCS, тестованість компонентів та легку інтеграцію зі стороннім ПЗ [20].

Архітектура Airflow складається з декількох ключових компонентів: Scheduler (відповідає за планування виконання завдань), Worker-вузли (виконують завдання), Metadata Database (зберігає стан та метадані), і Web UI (моніторинг та керування). Згідно зі звітом «2024 State of Apache Airflow» [21], 92% користувачів рекомендують платформу, 55% організацій використовують Airflow щоденно, а частка ML/AI-проектів у конвеєрах Airflow зросла на 24% рік до року.

Для цілей даної роботи принципово важливими є: механізм автоматичних повторних спроб (retries) при збоях завдань, параметризовані DAG за допомогою Param API та TaskFlow API (декоратори `@dag`, `@task`), що використані у розробленому конвеєрі.

1.4.2. Ollama та llama3.2

Ollama — відкрита інфраструктура для запуску великих мовних моделей на локальному обладнанні. Проект налічує понад 500 контрибutorів і більше 150 тисяч зірок на GitHub, що свідчить про його зрілість і широке прийняття в спільноті [22]. Ollama надає простий CLI та REST API (за замовчуванням на порту 11434), що дозволяє інтегрувати локальні LLM у будь-які додатки без необхідності звернення до хмарних провайдерів.

Технічно Ollama побудований на основі llama.cpp — висококоптимізованої C++-бібліотеки для інференсу LLM, що підтримує CPU та GPU виконання (CUDA, Metal, OpenCL), багатопоточність та SIMD-оптимізації. Моделі зберігаються у форматі GGUF (Grok-GGML Unified Format), що дозволяє ефективно запускати квантовані версії на обладнанні без дискретної відеокарти [23].

Порівняльне дослідження локально запусчених LLM [24] підтвердило, що Ollama є надійним і широко прийнятим середовищем для виконання і

порівняльного оцінювання відкритих LLM. Його підтримка квантованих моделей і структурована реєстрація ефективності роблять його придатним для виробничого розгортання.

Llama 3.2 — остання лінійка відкритих мовних моделей від Meta AI, оголошена у вересні 2024 року [25]. Текстові моделі розміром 1B та 3B параметрів мають контекстне вікно 128K токенів і є найкращими у своєму класі для пристроїв з обмеженими ресурсами (edge devices) у задачах узагальнення, виконання інструкцій та переписування тексту. Для потреб даної роботи обрано модель Llama3.2, яка надає оптимальний баланс між якістю аналізу та швидкістю локального інференсу.

Таблиця 1.4 – Технічні характеристики моделі Llama 3.2 у порівнянні з попередниками

Характеристика	LLaMA 2 (7B)	LLaMA 3 (8B)	LLaMA 3.2 (3B)	LLaMA 3.2 (1B)
Параметри	7B	8B	3B	1B
Контекстне вікно	4K токенів	8K токенів	128K токенів	128K токенів
Мультимодальність	Ні	Ні	Ні (текст)	Ні (текст)
Edge-оптимізація	Ні	Часткова	Так	Так
Zero-shot SA F1	~0.76	~0.81	~0.79	~0.72

1.4.3. Microsoft Blazor

Blazor — компонент ASP.NET Core від Microsoft, що дозволяє розробникам створювати інтерактивні веб-застосунки на C# замість JavaScript [26]. Фреймворк підтримує декілька моделей хостингу: Blazor Server (логіка виконується на сервері, зміни передаються через SignalR), Blazor WebAssembly (весь .NET-рантайм завантажується в браузер і виконується через WebAssembly), та гібридні підходи (Blazor Auto в .NET 8).

Blazor Architecture

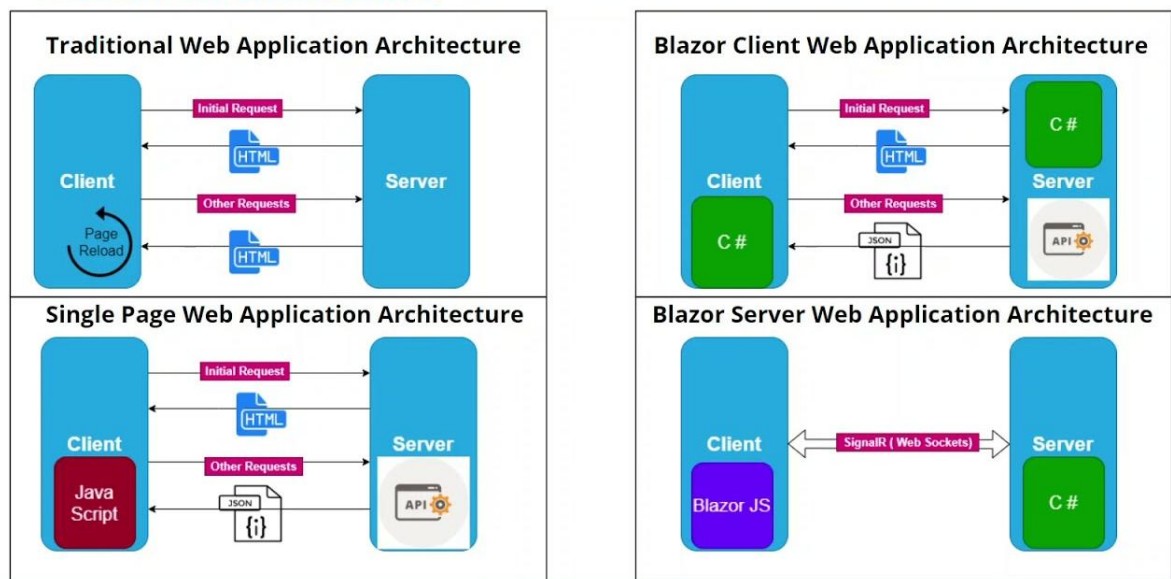


Рисунок 1.4 – Порівняння моделей хостингу Blazor

Прийняття Blazor у промисловості стрімко зростає: за даними BuiltWith, кількість живих вебсайтів на Blazor зросла з 12,5 тисяч у листопаді 2023 року до більш ніж 149 тисяч у серпні 2025 року, що відповідає десятикратному зростанню менш ніж за два роки [27]. На Microsoft Build 2025 компанія підтвердила Blazor SSR як пріоритетний напрям у розвитку .NET web UI.

Для організацій із розвиненою .NET-інфраструктурою Blazor є стратегічно вигідним вибором, оскільки дозволяє створювати повностековий застосунок на єдиній мові (C#) без необхідності підтримувати окремі кодові бази для фронтенду та бекенду [28]. У контексті даної роботи Blazor Server використовується для побудови моніторингового дашборду з відображенням результатів аналізу тональності та статистики самовідновлення.

1.5. Постановка задачі

Проведений огляд предметної галузі дозволяє сформулювати задачу даної бакалаврської роботи у такий спосіб. Необхідно розробити

самовідновлювальний конвеєр аналізу тональності відгуків, що відповідає таким вимогам:

По-перше, конвеєр повинен автоматично виявляти та виправляти типові дефекти вхідних даних (відсутній текст, хибний тип, порожній або надмірно довгий запис) без зупинки обробки та без втрати контексту виконання.

По-друге, система має підтримувати пакетну обробку довільного розміру з можливістю конфігурації через параметризовані запуски (`batch_size`, `offset`, `ollama_model`), що дозволяє гнучко масштабувати її під різні обсяги даних.

По-третє, для задачі аналізу тональності необхідно застосувати локальну велику мовну модель (llama3.2 через Ollama) замість хмарних провайдерів — з метою забезпечення конфіденційності даних, контрольованості вартості та можливості автономного розгортання.

По-четверте, конвеєр має генерувати структуровані health-звіти зі статусами HEALTHY / WARNING / DEGRADED / CRITICAL та зберігати детальну статистику: розподіл тональностей, кореляцію зірок і sentiment, розподіл дій відновлення, середню впевненість (`confidence`) за статусами.

По-п'яте, результати роботи конвеєра мають відображатися у веб-інтерфейсі на базі Blazor, що надає зручний інструмент для аналізу та аудиту виконаних запусків у реальному часі.

Таким чином, об'єктом дослідження є процес обробки й аналізу текстових даних у виробничих конвеєрах з урахуванням питань якості та відмовостійкості. Предметом дослідження є методи та інструменти побудови самовідновлювальних конвеєрів аналізу тональності на базі Apache Airflow та локальних LLM.

Науково-практичне значення роботи полягає у демонстрації комплексного підходу до інтеграції механізмів self-healing, LLM-інференсу

та веб-моніторингу в єдиній системі, що може слугувати зразком для аналогічних виробничих рішень у галузі DataOps та MLOps.

Висновки до розділу 1

У першому розділі проведено комплексний огляд теоретичних основ та технологічного стеку системи, що розробляється. Визначено, що якість даних є критичним чинником у реальних датасетах, а Yelp Academic Dataset є репрезентативним прикладом краудсорсингового набору даних з характерними типами дефектів. Систематизовано п'ять основних типів помилок вхідних даних та відповідні стратегії їх автоматичного виправлення.

Розглянуто концепцію self-healing у конвеєрах обробки даних, порівняно детерміністичні та навчальні підходи і обґрунтовано вибір детерміністичного підходу на основі правил для даної роботи. Здійснено огляд еволюції методів аналізу тональності — від лексичних словників до трансформерних LLM — та обґрунтовано використання моделі llama3.2.

Надано характеристику обраного технологічного стеку: Apache Airflow (оркестрація DAG-ів), Ollama (локальний LLM-інференс), llama3.2 (модель SA) та Blazor Server (веб-дашборд). На підставі аналізу сформульовано постановку задачі дослідження, визначено об'єкт і предмет дослідження.

РОЗДІЛ 2

ПРОЄКТУВАННЯ СИСТЕМИ

2.1. Загальна архітектура системи

Система Self-Healing Pipeline є багатокомпонентним програмним комплексом, що поєднує конвеєр обробки даних на базі Apache Airflow з аналітичною базою даних DuckDB та веб-дашбордом на Blazor. Архітектурно система складається з трьох відокремлених рівнів: рівня обробки даних (Python/Airflow), рівня зберігання (DuckDB, JSON-файли) та рівня відображення (ASP.NET Core + Blazor WASM) [19].

На рисунку 2.1 зображено загальну архітектуру системи та потік даних між її компонентами.

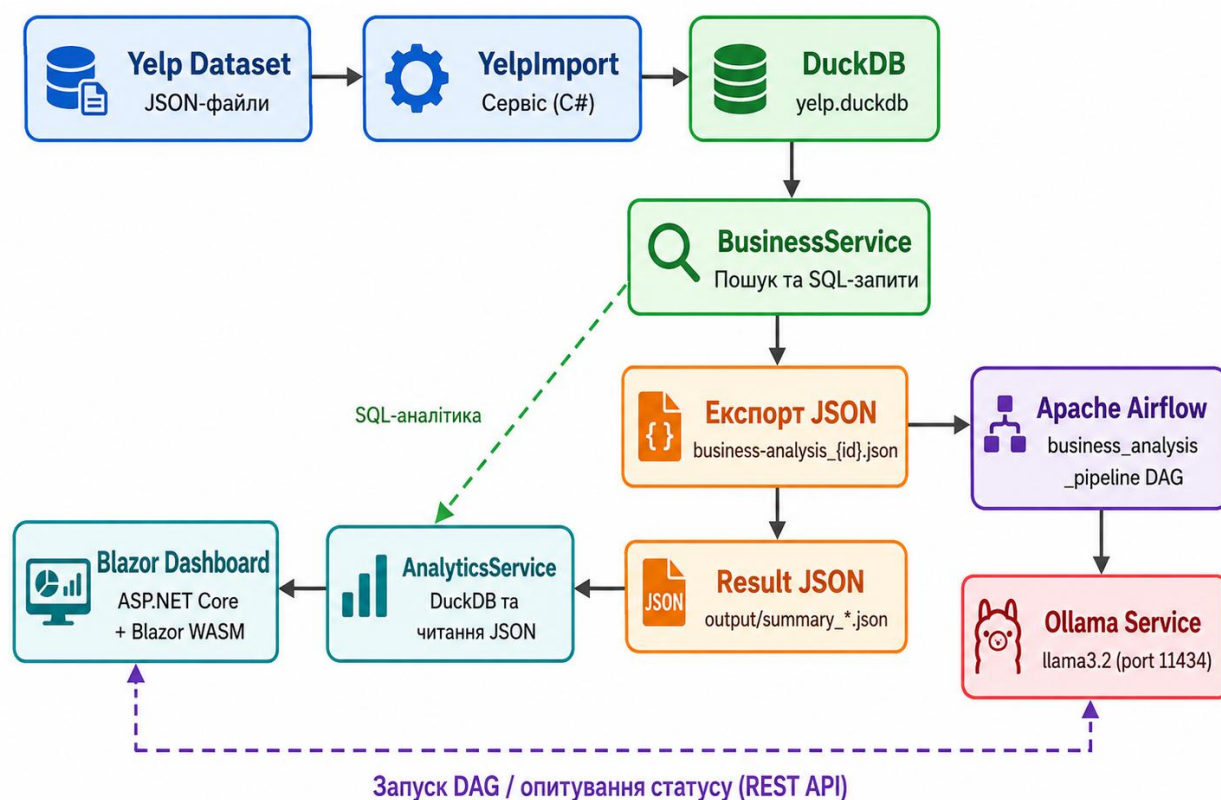


Рисунок 2.1 – Загальна архітектура системи Self-Healing Pipeline

Потік даних системи включає п'ять послідовних етапів. На першому етапі сирі JSON-файли Yelp Academic Dataset (загальним обсягом понад 6 ГБ) імпортуються у колонкову аналітичну базу даних DuckDB через сервіс YelpImportService. На другому етапі BusinessService виконує SQL-запити до DuckDB для пошуку бізнес-сутностей та генерує файл у форматі NDJSON для конкретного бізнесу. На третьому етапі Airflow DAG здійснює пакетну обробку відгуків: діагностику та самовідновлення даних, аналіз тональності через Ollama/llama3.2 та агрегацію результатів у JSON-файл. На четвертому етапі AnalyticsService зчитує як результуючий JSON конвеєру, так і безпосередньо DuckDB для розширеної аналітики. П'ятий етап — відображення даних у Blazor WASM-дашборді.

Ключовим архітектурним рішенням є використання DuckDB як центрального аналітичного сховища замість прямої роботи з сирими JSON-файлами. DuckDB — це вбудована OLAP-СУБД зі стовпчастим форматом зберігання та векторизованим виконанням запитів, що оптимізована для аналітичних робочих навантажень на звичайному апаратному забезпеченні [29]. Завдяки цьому пошукові запити по 6+ ГБ датасету виконуються за мілісекунди замість хвилин при прямому читанні JSON-файлів [30].

Таблиця 2.1 – Порівняльний аналіз підходів до зберігання даних

Критерій	Сирі JSON-файли	SQLite (row-based)	DuckDB (columnar)
Пошук по 6 ГБ	Хвилини (sequential scan)	Секунди (B-tree index)	Мілісекунди (zoned maps)
Аналітичні запити	Потребує завантаження в пам'ять	Повільно (row scan)	Швидко (column scan)
Агрегації (COUNT, AVG)	Мануальний код	Задовільно	Оптимізовано нативно
Out-of-core обробка	Ні	Часткова	Так (spill to disk)
Паралелізм запитів	Ні	Ні	Так (multi-core)
Розгортання	Просте	Просте	Простий .duckdb файл

Ще одним ключовим рішенням є контейнеризація компонента Airflow за допомогою Docker Compose. Контейнеризація забезпечує ізоляцію залежностей, відтворюваність середовища та спрощення розгортання на будь-якій операційній системі [31]. Сервіс Ollama запускається на хост-машині і доступний з Docker-контейнера через адресу `host.docker.internal:11434`, що дозволяє ефективно використовувати GPU хост-машини без прокидання GPU у контейнер.

2.2. Проєктування рівня зберігання даних (DuckDB)

Рівень зберігання даних включає два типи сховищ: аналітичну базу DuckDB та файлову систему JSON-файлів. DuckDB використовується для зберігання вихідного Yelp датасету (5 таблиць), тоді як результати роботи конвеєра зберігаються у JSON-файлах у директорії `output/` і зчитуються напряму через .NET-сервіси.

Процес імпорту до DuckDB здійснюється через `YelpImportService`, який послідовно читає п'ять NDJSON-файлів Yelp та завантажує їх у відповідні таблиці за допомогою `DuckDB.NET` — офіційного .NET-провайдера DuckDB. Підтримка прямого читання JSON DuckDB дозволяє виконувати SQL-запит вигляду `INSERT INTO business SELECT * FROM read_json_auto('...')` без проміжної десеріалізації в об'єкти C# [32].

Таблиця 2.2 – Структура таблиць DuckDB (Yelp Academic Dataset)

Таблиця	Вихідний файл	Ключові поля	Призначення в системі
business	yelp_academic_dataset_business.json	business_id, name, city, state, stars, review_count, categories	Пошук і вибір бізнесу в дашборді
review	yelp_academic_dataset_review.json	review_id, business_id, user_id, stars, text, date	Експорт відгуків для конкретного бізнесу
user	yelp_academic_dataset_user.json	user_id, name, review_count	Аналітика топ-рецензентів
tip	yelp_academic_dataset_tip.json	business_id, user_id, text, date	Розширена аналітика
checkin	yelp_academic_dataset_checkin.json	business_id, date	Heatmap активності

Результати роботи конвеєра зберігаються у файлах формату `sentiment_analysis_summary_{timestamp}_Offset{N}.json` у директорії `output/`. Ці файли зчитуються `PipelineRunsController` та `AnalyticsService` для відображення в дашборді. Така дворівнева архітектура зберігання (`DuckDB` для вихідних даних + `JSON` для результатів) забезпечує незалежність модулів: конвеєр `Airflow` не залежить від `Blazor`-бекенду і може функціонувати автономно.

2.3. Проєктування конвеєрів `Apache Airflow`

Система містить два незалежних `DAG`-и (`Directed Acyclic Graph`), кожен з яких вирішує власну задачу. Вибір між двома конвеєрами зумовлений різними сценаріями використання: масова обробка всього датасету та цільовий аналіз одного конкретного бізнесу.

Таблиця 2.3 – Порівняльна характеристика двох `DAG`-ів системи

Характеристика	<code>self_healing_pipeline</code>	<code>business_analysis_pipeline</code>
<code>dag_id</code>	<code>self_healing_pipeline</code>	<code>business_analysis_pipeline</code>
Призначення	Масова обробка всього датасету	Аналіз відгуків одного бізнесу
Вхідний файл	<code>yelp_academic_dataset_review.json</code>	<code>business-analysis_{id}.json</code>
<code>DEFAULT batch_size</code>	100 записів	0 (всі записи)
<code>retries</code>	2	1
<code>execution_timeout</code>	30 хвилин	2 години
<code>tags</code>	<code>sentiment_analysis</code> , <code>nlp</code> , <code>ollama</code> , <code>yelp_reviews</code>	<code>business_analysis</code> , <code>sentiment</code> , <code>ollama</code> , <code>yelp</code>
Запуск	Вручну або <code>batch_runner.py</code>	Через <code>Blazor</code> (<code>BusinessAnalysis.razor</code>)

Обидва `DAG`-и реалізують однакову шестикрокову послідовність задач з використанням `TaskFlow API` (декоратори `@dag` та `@task`), що є рекомендованим підходом в `Apache Airflow 3.x` для визначення залежностей через передачу повернених значень між задачами [20]. Граф задач зображено на рисунку 2.2.

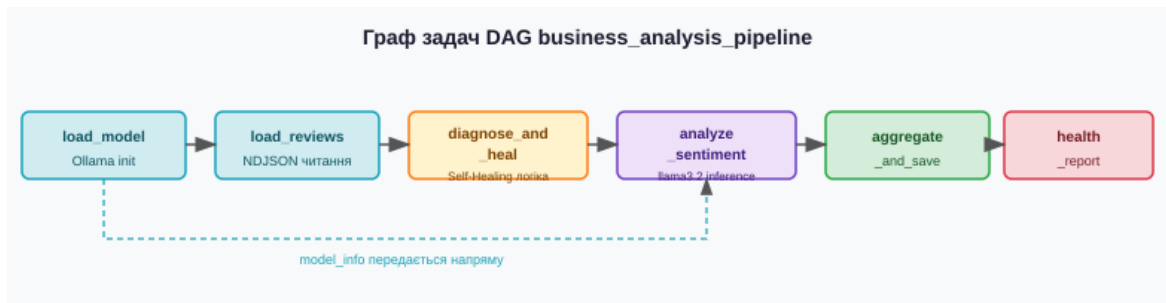


Рисунок 2.2 – Граф задач DAG business_analysis_pipeline

Параметризація DAG-ів реалізована через Param API, що дозволяє передавати конфігурацію при запуску через UI Airflow або REST API. Параметр `input_file` визначає шлях до вхідного файлу всередині контейнера (`/opt/airflow/input/`), `batch_size` і `offset` забезпечують пакетну обробку великих файлів, а `ollama_model` дозволяє перемикатися між моделями без зміни коду DAG-у.

Для масштабованої обробки великих датасетів (мільйони записів) передбачено скрипт `batch_runner.py`, що автоматично розбиває датасет на пакети і запускає кілька DAG-ів послідовно або паралельно. Використання `ThreadPoolExecutor` дозволяє одночасно обробляти `N` пакетів, що у `N` разів скорочує час обробки при достатніх апаратних ресурсах.

2.4. Проєктування модуля самовідновлення

Модуль самовідновлення реалізований у вигляді задачі `diagnose_and_heal_batch` DAG-у і є центральним компонентом системи. Він відповідає за діагностику кожного вхідного запису та застосування відповідної стратегії виправлення без переривання обробки.

Алгоритм функції `_heal_review` виконує послідовну перевірку п'яти умов для кожного відгуку і застосовує перше спрацьоване правило виправлення. Блок-схема алгоритму зображена на рисунку 2.3. Важливо, що алгоритм є детерміністичним: для кожної комбінації вхідних даних результат

виправлення є однозначним і відтворюваним, що забезпечує аудитабельність системи.

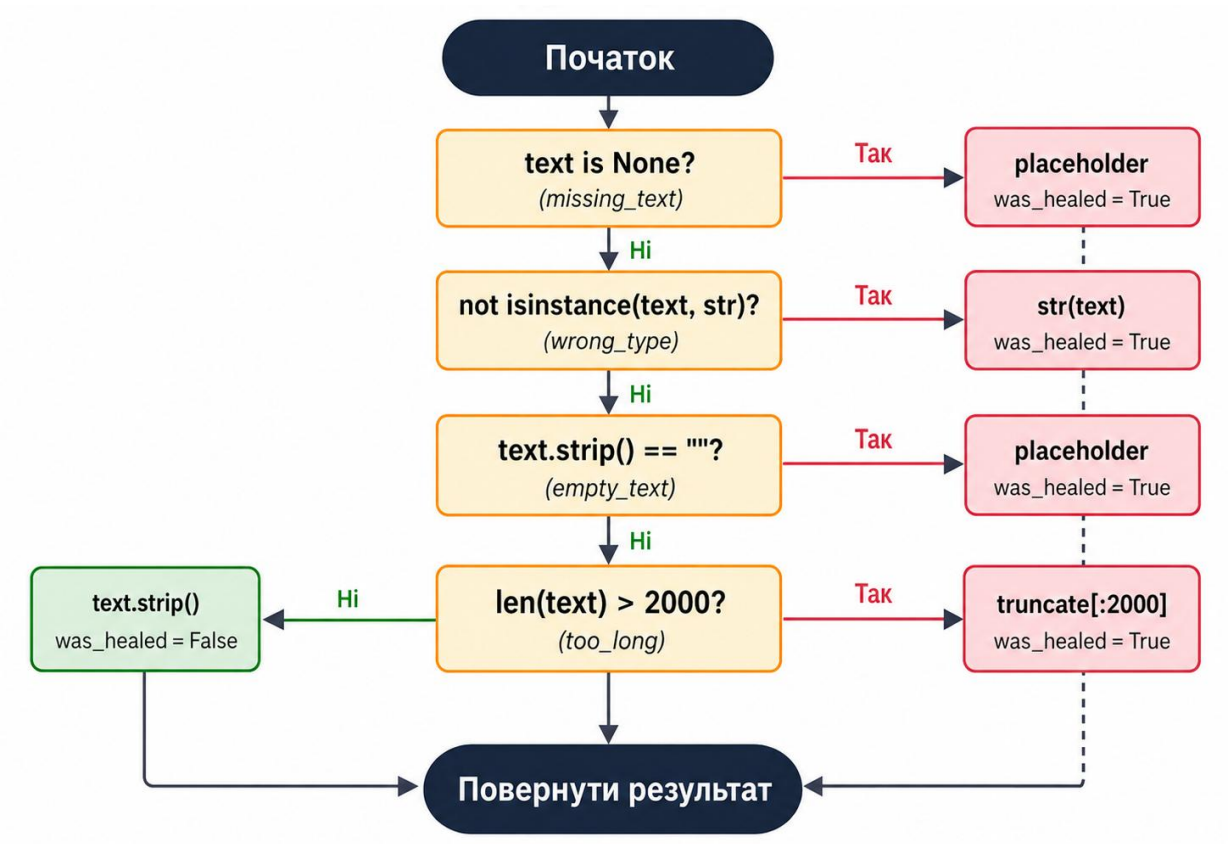


Рисунок 2.3 – Блок-схема алгоритму самовідновлення `_heal_review`

Таблиця 2.4 – Специфікація типів помилок та стратегій відновлення

Тип помилки	Умова виявлення	Стратегія виправлення	healed_text	Пріоритет
missing_text	text is None	fill_with_placeholder	"No review text provided."	1 (найвищий)
wrong_type	not isinstance(text, str)	type_conversion → str(text)	str(text).strip()	2
empty_text	not text.strip()	fill_with_placeholder	"No review text provided."	3
special_characters_only	not re.search(r"[a-zA-Z0-9]", text)	replaced_special_characters	"[Non-text content]"	4
too_long	len(text) > MAX_TEXT_LENGTH (2000)	truncated_text	text[:1997] + "..."	5
—	Жодна умова не спрацювала	none	text.strip()	—

У разі недоступності Ollama (мережева помилка, timeout) активується режим деградованої обробки через функцію `_degraded_results`: всі записи поточного пакету отримують статус `degraded` та нейтральний прогноз тональності (`NEUTRAL`, `confidence=0.5`). Це забезпечує принцип *graceful degradation* — конвеєр завершується успішно навіть при відмові зовнішнього сервісу, а ступінь деградації відображається у `health`-звіті [12].

2.5. Проєктування модуля аналізу тональності

Модуль аналізу тональності реалізований у задачі `analyze_sentiment` (`business_analysis_pipeline`) та `batch_analyze_sentiment` (`self_healing_pipeline`) і відповідає за взаємодію з Ollama REST API для класифікації тональності кожного відгуку.

Промпт-інжиніринг є критично важливим аспектом взаємодії з LLM. Для забезпечення структурованої та парсбельної відповіді модель `llama3.2` інструктується повертати виключно JSON-об'єкт у форматі `{"sentiment": "POSITIVE|NEGATIVE|NEUTRAL", "confidence": 0.0-1.0}`. Така структура промпту відповідає рекомендаціям щодо *prompt engineering* для задач класифікації тексту з LLM [16].

Для кожного запиту до Ollama передбачено три спроби (`OLLAMA_RETRIES=3`) з `timeout 120` секунд (`OLLAMA_TIMEOUT=120`). При неможливості отримати відповідь після трьох спроб запис переводиться у стан `degraded`. Парсинг відповіді реалізований з подвійним *fallback*: спочатку виконується спроба розібрати JSON (враховуючи можливий ````json markdown wrapper`), а при невдачі — пошук ключових слів `POSITIVE/NEGATIVE` у сирому тексті відповіді.

У таблиці 2.5 наведено детальну специфікацію взаємодії програмної системи з Ollama REST API, яка використовується для аналізу емоційної тональності текстових відгуків. Описані параметри охоплюють мережеву

адресу сервісу, формат HTTP-запиту, конфігурацію мовної моделі та структуру промπτу, що поєднує текст відгуку з інструкцією щодо повернення структурованої відповіді у форматі JSON. Такий підхід забезпечує уніфікований інтерфейс інтеграції з локально розгорнутою великою мовною моделлю та спрощує подальшу автоматизовану обробку результатів аналізу.

Таблиця 2.5 – Специфікація взаємодії з Ollama REST API

Параметр	Значення / Опис
Endpoint	POST http://localhost:11434/api/chat
Модель	llama3.2 (конфігурується через OLLAMA_MODEL)
Формат запиту	JSON: {model, messages:[{role, content}]}
Структура промπτу	Аналіз тексту відгуку + інструкція повернути JSON з полями sentiment та confidence
Очікувана відповідь	{"sentiment":"POSITIVE NEGATIVE NEUTRAL","confidence":0.0-1.0}
Fallback парсинг	Пошук рядків POSITIVE/NEGATIVE/NEUTRAL у text-відповіді
Retry логіка	3 спроби, timeout 120 сек, деградація при вичерпанні спроб
Деградований стан	predicted_sentiment=NEUTRAL, confidence=0.5, status=degraded

Окрему увагу приділено механізмам надійності та відмовостійкості взаємодії. У разі некоректної або неструктурованої відповіді моделі застосовується fallback-парсинг, що дозволяє визначити тональність шляхом пошуку ключових маркерів у тексті. Додатково реалізовано retry-логіку з обмеженням кількості спроб і тайм-аутом виконання запиту. При повному вичерпанні спроб система переходить у деградований стан, у якому встановлюються безпечні значення тональності та рівня впевненості, що дозволяє зберегти стабільність роботи загального конвеєра аналізу даних навіть за умов збоїв зовнішнього сервісу.

2.6. Проєктування структури вихідних даних та health-звіту

Кожен запуск конвеєру генерує структурований JSON-файл з повним набором метрик. Ця структура є вхідними даними для Blazor-дашборду і

проєктується з урахуванням вимог до відображення всіх необхідних діаграм та звітів (таблиця 2.6).

Таблиця 2.6 – Схема JSON-файлу результатів конвеєру

Поле верхнього рівня	Тип	Опис
run_info	Object	Метадані запуску: timestamp, batch_size, offset, input_file
totals	Object	Лічильники: processed, success, healed, degraded
rates	Object	Частки: success_rate, healing_rate, degradation_rate (0.0–1.0)
sentiment_distribution	Object	Розподіл: POSITIVE, NEGATIVE, NEUTRAL (абсолютні значення)
healing_statistics	Dict<string,int>	Частота дій відновлення: {action_name: count}
star_sentiment_correlation	Dict<string,Object>	Correlation зірок і sentiment: {stars: {POSITIVE,NEGATIVE,NEUTRAL}}
average_confidence	Object	Середня впевненість за статусами: {success, healed, degraded}
results	Array<ReviewResult>	Масив всіх оброблених відгуків з деталями

Таблиця 2.7 – Логіка визначення health-статусу конвеєру

Статус	Умова	Значення degraded_rate	Значення healing_rate	Колір в UI
HEALTHY	Немає деградації, мало відновлень	= 0.0	≤ 50%	Зелений
WARNING	Багато відновлень, немає деградації	= 0.0	> 50%	Жовтий
DEGRADED	Є деградовані записи	$0 < d \leq 10\%$	будь-яке	Помаранчевий
CRITICAL	Значна деградація	> 10%	будь-яке	Червоний

Health-статус конвеєру (таблиця 2.7) визначається на основі двох показників: частки деградованих записів та частки вилікуваних записів від загального обсягу. Логіка визначення статусу реалізована у задачі health_report і відповідає концепції багаторівневих перевірок здоров'я системи [13].

2.7. Проєктування Blazor-дашборду

Веб-дашборд реалізований за моделлю Blazor Hosted WebAssembly: ASP.NET Core-сервер (проєкт Dashboard) обслуговує REST API та статичні

ресурси, тоді як Blazor WASM-клієнт (проект Dashboard.Client) виконується безпосередньо в браузері. Така архітектура поєднує переваги серверної обробки (доступ до файлової системи, DuckDB, Airflow REST API) зі збагаченим клієнтським інтерфейсом на C# без JavaScript [26].

Архітектуру Blazor-додатку зображено на рисунку 2.4.

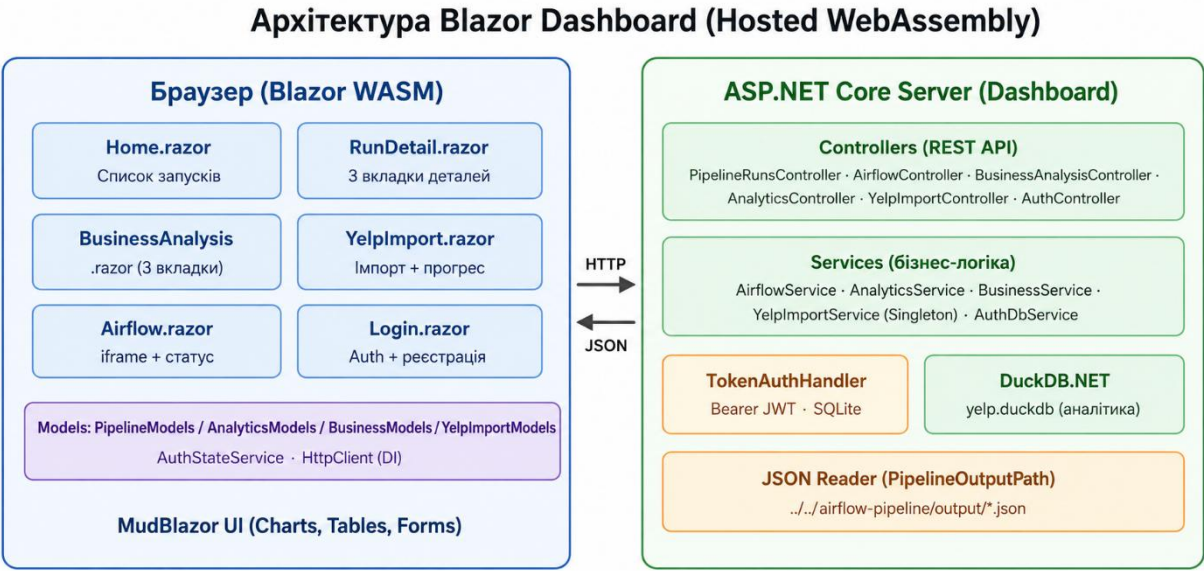


Рисунок 2.4 – Архітектура Blazor Hosted WebAssembly Dashboard

Таблиця 2.8 – Специфікація сторінок клієнтського застосунку Blazor

Маршрут	Компонент	Призначення	Ключові UI-компоненти
/login	Login.razor	Автентифікація та реєстрація користувача	MudTextField, MudButton, AuthStateService
/	Home.razor	Перелік усіх запусків конвеєру з health-статусами	MudTable (сортування), MudChip (статус)
/run/{filename}	RunDetail.razor	3-вкладковий детальний перегляд одного запуску	MudTabs, MudChart (Pie/StackedBar), MudTable відгуків
/business-analysis	BusinessAnalysis.razor	3-вкладковий воркфлоу: вибір → запуск DAG → аналітика	MudTable, PeriodicTimer polling, SVG charts
/airflow	Airflow.razor	Вбудований iframe Airflow UI	iframe, перевірка доступності API
/files	FileTree.razor	Перегляд файлової структури output/	MudTreeView, FileTreeItem.razor
/yelp-import	YelpImport.razor	Імпорт Yelp JSON → DuckDB	MudProgressLinear, PeriodicTimer (1 сек polling)

Автентифікація реалізована через власний механізм Bearer-токенів: AuthController зберігає облікові записи у SQLite-базі auth.db через AuthDbService, видає токен при успішному вході, а TokenAuthHandler валідує токен у кожному запиті до захищених ендпоінтів. На клієнтській стороні AuthStateService зберігає токен у пам'яті сесії та додає заголовок Authorization до всіх HTTP-запитів.

Для відстеження прогресу виконання DAG у реальному часі застосовано механізм polling на базі PeriodicTimer з інтервалом 5 секунд. При зверненні до AirflowController він проксує запити до Airflow REST API v2, отримуючи стан DAG run, стани окремих тасків та логи задачі analyze_sentiment. Парсинг логів дозволяє відображати поточний прогрес обробки відгуків у вигляді прогрес-бара.

Таблиця 2.9 – Ключові REST API ендпоінти серверного застосунку

Метод	Маршрут	Сервіс	Призначення
GET	/api/pipelinerruns	PipelineRunsController	Список усіх JSON-файлів результатів
GET	/api/pipelinerruns/{filename}	PipelineRunsController	Повний вміст одного результату
GET	/api/airflow/status	AirflowController	Перевірка доступності Airflow
POST	/api/airflow/trigger-dag	AirflowController	Запуск business_analysis_pipeline
GET	/api/airflow/dag-run/{runId}	AirflowController	Стан DAG run
GET	/api/airflow/task-instances/{runId}	AirflowController	Стани усіх тасків
GET	/api/airflow/task-logs/{runId}/{taskId}	AirflowController	Логи + прогрес аналізу
GET	/api/business-analysis/businesses	BusinessAnalysisController	Пошук бізнесів у DuckDB
POST	/api/business-analysis/export/{id}	BusinessAnalysisController	Експорт відгуків → NDJSON
GET	/api/analytics/summary/{businessId}	AnalyticsController	Зведена аналітика по бізнесу
POST	/api/yelp-import/start	YelpImportController	Запуск імпорту до DuckDB
POST	/api/auth/login	AuthController	Автентифікація, видача токена

Компонент BusinessAnalysis.razor є найскладнішим у системі і реалізує повний воркфлоу у трьох вкладках MudTabs. Перша вкладка «Вибір бізнесу» забезпечує пошук по DuckDB з debounce 400 мс та відображення деталей вибраного бізнесу. Кнопка «Витягнути відгуки» викликає POST /api/business-analysis/export/{id}, що генерує NDJSON-файл у директорії input/. Друга вкладка «Запуск DAG» доступна тільки після успішного експорту та надає форму параметрів запуску (модель, batch_size, offset) і відображає прогрес виконання через polling. Третя вкладка «Аналітика» завантажує паралельно вісім аналітичних запитів до AnalyticsService та відображає результати у вигляді графіків, виконаних на MudChart та власних SVG-компонентах (heatmap 7×24 годин, scatter-plot, horizontal bar chart).

Висновки до розділу 2

У другому розділі спроектовано архітектуру системи Self-Healing Pipeline та всіх її компонентів. Обґрунтовано використання DuckDB як колонкового аналітичного сховища, що забезпечує субсекундний час виконання аналітичних запитів до багатогігабайтного датасету Yelp порівняно з прямим читанням JSON-файлів. Визначено дворівневу архітектуру зберігання: DuckDB для вихідних даних та JSON-файли для результатів роботи конвеєру.

Спроектовано два DAG-и Apache Airflow (self_healing_pipeline та business_analysis_pipeline) з однаковою шестикроковою структурою задач, що відрізняються призначенням, параметрами та сценаріями запуску. Формалізовано алгоритм самовідновлення у вигляді блок-схеми з п'ятьма класами помилок та відповідними стратегіями виправлення. Описано механізм graceful degradation при недоступності Ollama.

Спроектовано Blazor Hosted WebAssembly дашборд з розподілом на сервер (ASP.NET Core + 7 контролерів + 5 сервісів) і клієнт (7 сторінок + MudBlazor UI). Визначено механізм аутентифікації через Bearer-токени та

SQLite, а також real-time polling через PeriodicTimer для відстеження прогресу DAG. Визначено JSON-схему вихідних даних конвеєру та логіку

РОЗДІЛ 3

РЕАЛІЗАЦІЯ СИСТЕМИ

3.1. Технологічний стек та середовище розробки

Реалізація системи Self-Healing Pipeline здійснювалась на двох незалежних підсистемах: конвеєрі обробки даних (Python/Airflow) та веб-дашборді (C#/.NET). Обидві підсистеми розміщені в одному монорепозіторії, що спрощує спільне розгортання та управління версіями. Для розробки використовувалися Visual Studio Code, PyCharm та Claude Code як AI-асистент (конфігурація задокументована у `.claude/settings.local.json`) [22].

Таблиця 3.1 – Технологічний стек системи

Компонент	Технологія	Версія	Призначення
Оркестрація	Apache Airflow	3.0+	Визначення, планування та виконання DAG-ів
LLM inference	Ollama + llama3.2	$\geq 0.6.0$ / Sep 2024	Локальний аналіз тональності
Аналітична БД	DuckDB (DuckDB.NET)	1.0+ / .NET binding	Зберігання Yelp датасету, SQL-аналітика
Auth БД	SQLite (Microsoft.Data.Sqlite)	LTS	Облікові записи та Bearer-токени
Веб-фреймворк	ASP.NET Core 8	.NET 8 LTS	REST API сервер
Frontend	Blazor Hosted WebAssembly	.NET 8	SPA-клієнт на C#
UI бібліотека	MudBlazor	$\geq 6.x$	Компоненти Material Design
Контейнеризація	Docker + Docker Compose	3.8	Ізольоване середовище Airflow
Мова (pipeline)	Python	3.12+	DAG-и, self-healing логіка
Мова (dashboard)	C# 12	.NET 8	Серверні сервіси та Blazor компоненти

Структура репозиторію організована за модульним принципом: директорія `airflow-pipeline/` містить усі Python-компоненти (DAG-и, Docker Compose, `requirements.txt`, скрипти), а `blazor-dashboard/` — повноцінний .NET solution з двома проєктами (Dashboard та Dashboard.Client). Така структура дозволяє незалежно розгорнути та тестувати кожну підсистему.

3.2. Реалізація рівня зберігання даних

3.2.1. Імпорт Yelp даних до DuckDB

Клас `YelpImportService` реалізований як `Singleton` (zareєстрований через `AddSingleton` у `Program.cs`), що гарантує єдиний стан імпорту впродовж усього часу роботи серверного застосунку. Це застосування патерну `Singleton` забезпечує коректну роботу механізму скасування (`CancellationTokenSource`) та спільний стан прогресу між кількома HTTP-запитами від клієнта [33].

Ключовою особливістю реалізації є використання нативної можливості `DuckDB` читати JSON-файли безпосередньо через функцію `read_json_auto()` без попередньої десеріалізації в `C#`-об'єкти. Наведений нижче фрагмент показує, як масив таблиць → файл відображається у послідовне виконання `INSERT`:

```
// YelpImportService — вставка з NDJSON напряму через DuckDB
cmd.CommandText = $"""
    INSERT INTO {table}
    SELECT * FROM read_json_auto('{filePath}',
format='newline_delimited');
""";
```

Метод `EnsureSchema()` при першому запуску автоматично створює таблиці через `CREATE TABLE IF NOT EXISTS`, забезпечуючи ідемпотентність розгортання. Повна реалізація методу імпорту наведена у Додатку А.1.

3.2.2. *BusinessService та AnalyticsService*

`BusinessService` відповідає за два основних сценарії: пошук бізнесів у `DuckDB` через `ILIKE`-запит та експорт відгуків у `NDJSON`-файл. `NDJSON` обрано замість стандартного `JSON`-масиву, оскільки `Airflow` читає файл рядково через `itertools.islice()`, що дозволяє ефективно реалізувати батчинг без завантаження всього файлу у пам'ять. Метод `SaveReviewsToJsonAsync()`

серіалізує кожен запис через StreamWriter з SnakeCaseLower-конвенцією імен полів (Додаток А.2).

AnalyticsService поєднує два джерела даних: JSON-файл конвеєру (для тональності, confidence, healing statistics) та DuckDB (для часових трендів, heatmap, топ-рецензентів). Паралельне завантаження восьми аналітичних ендпоінтів реалізовано через Task.WhenAll(), що скорочує час відповіді до максимуму з восьми запитів:

```
// Паралельний запит 8 ендпоінтів одночасно
await Task.WhenAll(t1, t2, t3, t4, t5, t6, t7, t8);
// Час відповіді = max(t1..t8), а не sum(t1..t8)
```

Повні SQL-запити AnalyticsService (heatmap, тренди, топ-рецензентів) наведені у Додатку А.3.

3.3. Реалізація конвеєрів Apache Airflow

3.3.1. TaskFlow API та передача даних між задачами

Обидва DAG-и реалізовані з використанням TaskFlow API (Airflow 2.0+), що є рекомендованим підходом для Python-нативних конвеєрів [20]. Декоратор @dag перетворює функцію на DAG-фабрику, а @task — на XComArg, що автоматично серіалізує повернені значення та передає їх залежним задачам. Залежності стають явними з сигнатури функції, без XCom.push()/pull(). Ключовий фрагмент визначення DAG:

```
model_info = load_model()          # повертає dict
reviews    = load_reviews()
healed     = diagnose_and_heal(reviews)
analyzed   = analyze_sentiment(healed, model_info) # явна залежність
summary    = aggregate_and_save(analyzed)
_          = health_report(summary)
```

Повне визначення DAG з усіма параметрами та задачами наведено у Додатку Б.1.

3.3.2. Параметризація та *batch_runner.py*

Param API Airflow дозволяє визначати типізовані параметри з описом та значеннями за замовчуванням. Параметри `input_file`, `batch_size`, `offset` та `ollama_model` забезпечують повну конфігурованість без зміни коду DAG. Прапор `render_template_as_native_obj=True` забезпечує коректну десеріалізацію Python-типів з JSON-контексту Airflow.

Скрипт `batch_runner.py` реалізує паралельний запуск через `ThreadPoolExecutor`. При `parallel=N` одночасно запускається N DAG-ів з різними `offset`-значеннями. Код скрипту наведено у Додатку Б.2.

3.4. Реалізація модуля самовідновлення

Функція `_heal_review` реалізує патерн «Ланцюжок відповідальності» (Chain of Responsibility) [33]: кожна умова перевіряє конкретний тип дефекту і при спрацюванні застосовує стратегію виправлення, не передаючи обробку далі. Це гарантує однозначність: кожен запис отримує рівно одну дію відновлення.

Перед будь-яким виправленням оригінальний текст зберігається у полі `original_text`, що дозволяє дашборду відображати порівняння «до / після» у вкладці Healing Report. Повна реалізація функції наведена у Додатку В.1. Скорочений фрагмент логіки ланцюжка:

```
if text is None:
    result.update(error_type="missing_text", healed_text="No review text
provided.", was_healed=True)
elif not isinstance(text, str):
    result.update(error_type="wrong_type",
healed_text=str(text).strip(), was_healed=True)
elif not text.strip():
    result.update(error_type="empty_text",   healed_text="No review text
provided.", was_healed=True)
elif not re.search(r"[a-zA-Z0-9]", text):
    result.update(error_type="special_characters_only", healed_text="[Non-
text content]", was_healed=True)
elif len(text) > Config.MAX_TEXT_LENGTH:
```

```
result.update(error_type="too_long",
healed_text=text[:MAX_TEXT_LENGTH-3]+"...", was_healed=True)
else:
    result["healed_text"] = text.strip() # нормальний шлях
```

Функція `_degraded_results` реалізує Null Object патерн для сценарію недоступності Ollama: замість виключення вона повертає список результатів з фіксованими значеннями (NEUTRAL, confidence=0.5, status=degraded). Реалізацію наведено у Додатку В.2.

3.5. Реалізація модуля аналізу тональності

Функція `_analyze_with_ollama` реалізує retry-логіку з трьома спробами для кожного відгуку (OLLAMA_RETRIES=3). При невдачі однієї спроби система продовжує до наступної. Тільки після вичерпання всіх спроб запис отримує статус degraded.

Промпт для llama3.2 явно вимагає JSON-відповідь з полями sentiment та confidence. Параметр temperature=0.1 зменшує варіативність відповідей, що підвищує стабільність парсингу. Логування прогресу кожні 10 записів (f'Processed {idx+1}/{total} reviews') використовується AirflowService на сервері для відображення прогрес-бару в UI. Повна реалізація функції наведена у Додатку В.3.

Функція `_parse_ollama_response` реалізує триступеневий парсинг: спочатку видалення markdown-обгортки, потім JSON-десеріалізація, і нарешті fallback через пошук ключових слів. Реалізацію наведено у Додатку В.4.

3.6. Реалізація автентифікації та авторизації

Система автентифікації реалізована як власний Bearer-токен механізм поверх ASP.NET Core Authentication middleware без використання JWT-бібліотек. Такий підхід обрано з міркувань простоти розгортання та

достатньої безпеки для локального середовища. Sequence-діаграма процесів логіну та авторизованого запиту зображена на рисунку 3.1.



Рисунок 3.1 – Sequence-діаграма автентифікації та авторизації

3.6.1. AuthDbService — зберігання облікових даних

AuthDbService використовує SQLite та підтримує дві таблиці: users (id, username, hash, salt, created) та tokens (token, user_id, username, created, expires). Хешування паролів реалізовано через PBKDF2-SHA256 зі 100 000 ітерацій та 32-байтною криптографічною сіллю, що відповідає рекомендаціям OWASP [34]. Токен генерується як 48 байт CSPRNG у Base64 (~384 біт ентропії), діє 30 днів:

```
private static string Hash(string password, byte[] salt) =>
    Convert.ToBase64String(Rfc2898DeriveBytes.Pbkdf2(
        password, salt, iterations: 100_000, HashAlgorithmName.SHA256,
        outputLength: 32));

var rawToken = Convert.ToBase64String(RandomNumberGenerator.GetBytes(48));
var expires = DateTime.UtcNow.AddDays(30);
```

Повна реалізація `AuthDbService` (включаючи `RegisterAsync`, `LoginAsync`, `ValidateTokenAsync`, `RevokeTokenAsync`) наведена у Додатку Г.1.

3.6.2. TokenAuthHandler та AuthController

`TokenAuthHandler` успадковує `AuthenticationHandler` та перевизначає `HandleAuthenticateAsync()`. Обробник зчитує заголовок `Authorization`, перевіряє токен через `AuthDbService` та формує `ClaimsPrincipal`. `AuthController` надає чотири ендпоінти: `POST /register`, `POST /login (AllowAnonymous)`, `POST /logout` та `GET /me (Authorize)`. Повні реалізації наведено у Додатку Г.2 та Г.3.

3.6.3. AuthStateService та MainLayout (Blazor WASM)

На стороні WASM автентифікацією керує `AuthStateService`. Метод `InitializeAsync()` викликається з `MainLayout.OnInitializedAsync()` і перевіряє збережений у `localStorage` токен через `GET /api/auth/me`. Компонент `MainLayout.razor` реалізує захист маршрутів: до ініціалізації показує `spinner`, після — або відображає застосунок (`IsAuthenticated=true`) або перенаправляє на `/login`:

```
// MainLayout.razor — захист маршрутів
@if (!Auth.Initialized) { <MudProgressCircular /> }
else if (!Auth.IsAuthenticated) { @Body /* тільки /login */ }
else { <MudLayout>@Body</MudLayout> }
```

Повні реалізації `AuthStateService` та `MainLayout` наведено у Додатку Г.4.

3.7. Реалізація Blazor-дашборду

3.7.1. UML-діаграма класів серверного проєкту

На рисунку 3.2 зображено UML-діаграму класів серверного проєкту `Dashboard` із залежностями між контролерами, сервісами та шаром моделей.

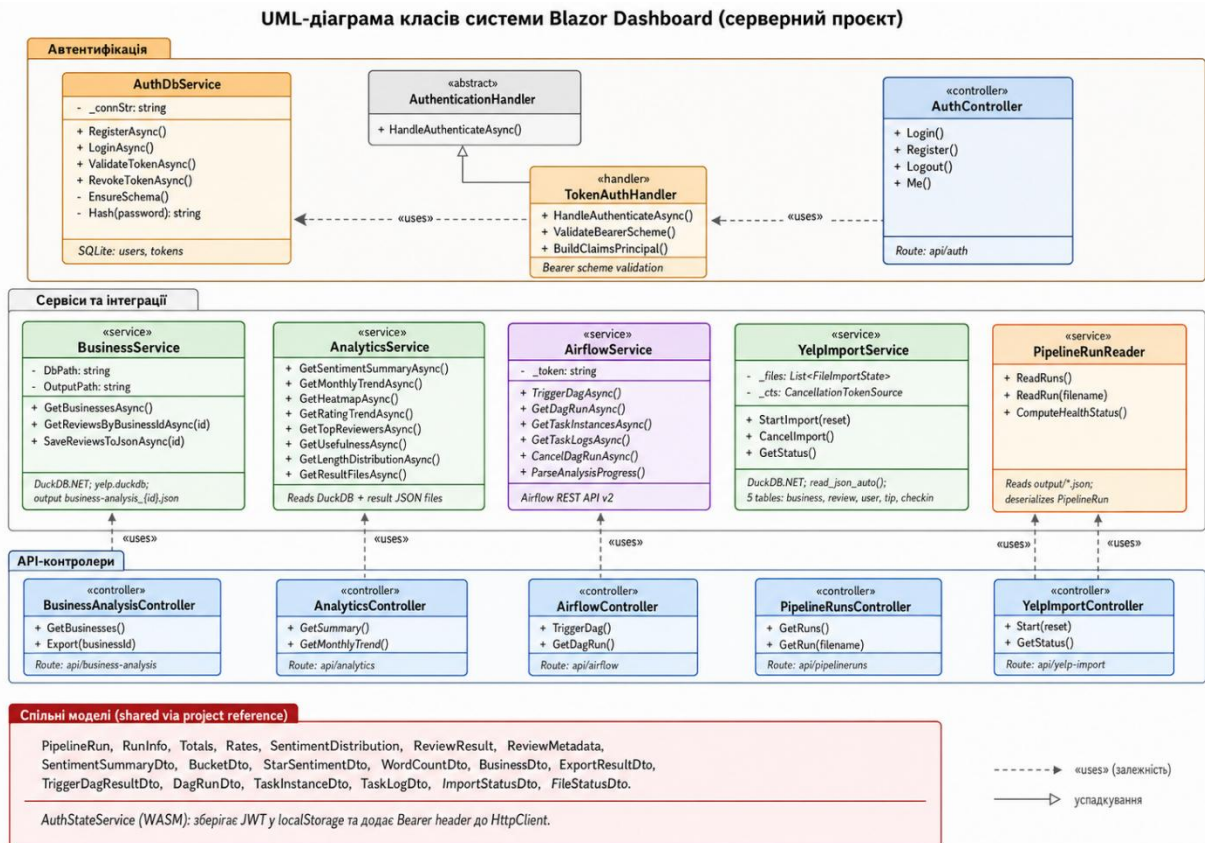


Рисунок 3.2 – UML-діаграма класів серверного проєкту Dashboard

Залежності між компонентами реалізовані через Dependency Injection контейнер ASP.NET Core. YelpImportService реєструється як Singleton, всі інші сервіси — як Scoped. Контролери отримують залежності через constructor injection. Shared-моделі (PipelineModels.cs, AnalyticsModels.cs та ін.) визначені у Dashboard.Client та доступні серверу через project reference, що унеможливорює дублювання DTO-класів. Реєстрацію сервісів у Program.cs наведено у Додатку Д.1.

3.7.2. AirflowService — інтеграція з Airflow REST API

AirflowService забезпечує взаємодію з Apache Airflow 3.x через REST API v2. Особливістю є кешування Bearer-токену автентифікації зі статичним SemaphoreSlim для потокобезпечного оновлення. Автентифікаційний пароль зчитується з файлу `simple_auth_manager_passwords.json.generated`, що

генерується Airflow при першому запуску. Метод ParseAnalysisProgress() витягує поточний прогрес з логів через регулярний вираз:

```
// Парсинг прогресу аналізу з логів Airflow
var matches = Regex.Matches(logs, @"Analyzed (\d+)/(\d+) reviews");
if (matches.Count > 0) {
    var last = matches[^1];
    return (int.Parse(last.Groups[1].Value),
    int.Parse(last.Groups[2].Value));
}
```

Повна реалізація AirflowService (TriggerDagAsync, GetDagRunAsync, GetTaskInstancesAsync, GetTaskLogsAsync, CancelDagRunAsync, GetTokenAsync) наведена у Додатку Д.2.

3.7.3. Механізм *real-time polling*

Відстеження прогресу виконання DAG реалізовано через PeriodicTimer — рекомендований у .NET 6+ інструмент для точних циклічних операцій. На рисунку 3.3 ілюструється послідовність взаємодій при polling.

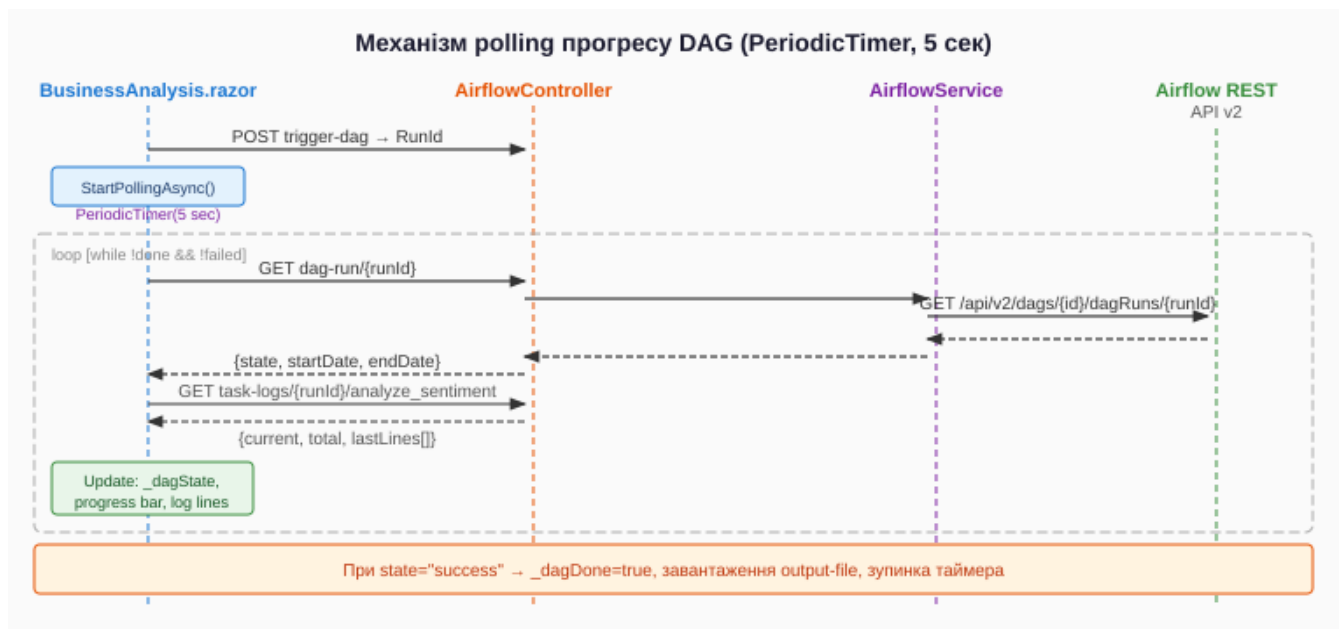


Рисунок 3.3 – Механізм *real-time polling* прогресу DAG через PeriodicTimer

Метод `StartPollingAsync()` запускається у фоновому Task після отримання `RunId`. Кожні 5 секунд виконуються запити стану DAG run, стану тасків та логів задачі `analyze_sentiment`. Реалізацію `StartPollingAsync()` та `PollOnceAsync()` наведено у Додатку Д.3.

3.7.4. SVG-візуалізація в дашборді

Для типів графіків, які не підтримуються нативно MudBlazor (heatmap 7×24, горизонтальний bar chart, scatter plot, top-words порівняння), реалізовані власні SVG-генератори у C#-кодi Blazor-компонентів. Методи типу `DrawHeatmapSvg()` повертають MarkupString. Наприклад, heatmap будується як двовимірний масив `int[7,24]`:

```
// Heatmap: заповнення сітки з даних DuckDB
int[,] grid = new int[7, 24];
foreach (var c in _heatmapData)
{
    int row = c.DayOfWeek == 0 ? 6 : c.DayOfWeek - 1;
    grid[row, c.Hour] = c.Count;
}
// Колір = лінійна інтерполяція між мінімумом та максимумом
double t = (double)count / maxCount;
string fill = $"rgb({17 + t*57:F0},{24 + t*198:F0},{39 + t*89:F0})";
```

Повну реалізацію SVG-генераторів (`DrawHeatmapSvg`, `DrawHealingBarSvg`, `DrawTopWordsSvg`) наведено у Додатку Д.4.

3.8. Розгортання системи

Компонент Apache Airflow розгортається у Docker-контейнері через `docker-compose.yml` у режимі standalone. Монтування поточної директорії як `/opt/airflow` забезпечує прямий доступ до DAG-ів без перезбірки образу [31]. Сервіс Ollama запускається на хост-машині та доступний з контейнера через `host.docker.internal:11434`. Blazor-дашборд запускається безпосередньо командою `dotnet run`. Конфігурацію `docker-compose.yml` та `appsettings.json` наведено у Додатку Е.

Висновки до розділу 3

У третьому розділі описано практичну реалізацію системи Self-Healing Pipeline. Модуль самовідновлення реалізує патерн «Ланцюжок відповідальності» для послідовної перевірки п'яти типів дефектів. Конвеєри Airflow побудовані на TaskFlow API з явною передачею даних між задачами. Рівень зберігання використовує DuckDB через DuckDB.NET та SQLite для облікових даних.

Власний механізм автентифікації базується на PBKDF2-SHA256 хешуванні паролів та Bearer-токенах зі строком дії 30 днів. Blazor-дашборд реалізує real-time polling через PeriodicTimer та власні SVG-візуалізації для графіків, що не підтримуються нативно MudBlazor.

РОЗДІЛ 4

ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

4.1. Методологія тестування

Тестування системи Self-Healing Pipeline проводилося за стратегією «чорного ящика» (black-box testing) та «сірого ящика» (grey-box testing). Стратегія чорного ящика застосовувалась для перевірки поведінки системи на рівні кінцевих результатів — вхідні дані подавались до системи, а результати порівнювались з очікуваними специфікаціями. Стратегія сірого ящика застосовувалась при тестуванні модулів, де внутрішня реалізація відома (наприклад, алгоритм `_heal_review`), що дозволило цілеспрямовано формувати тест-кейси для кожної гілки логіки.

Тестування охоплювало чотири рівні: модульне тестування окремих функцій (передусім `_heal_review` та `_parse_ollama_response`), інтеграційне тестування взаємодії між DAG-задачами, системне тестування повного циклу виконання конвеєру та функціональне тестування веб-інтерфейсу дашборду. Усі тест-кейси виконувались на реальному середовищі розгортання (Docker Compose + Ollama llama3.2 + Blazor Dashboard).

Таблиця 4.1 – Реєстр тест-кейсів системи

№	Модуль	Тест-кейс	Метод	Очікуваний результат
TC-01	Self-Healing	text = None	Black-box	missing_text, was_healed=True, placeholder
TC-02	Self-Healing	text = 12345 (int)	Grey-box	wrong_type, was_healed=True, "12345"
TC-03	Self-Healing	text = " " (пробіли)	Grey-box	empty_text, was_healed=True, placeholder
TC-04	Self-Healing	text = "!!!###@@@"	Grey-box	special_characters_only, was_healed=True
TC-05	Self-Healing	text довжиною 3000 символів	Grey-box	too_long, healed_text закінчується "..."
TC-06	Self-Healing	text = "Great food!" (коректний)	Grey-box	was_healed=False, status=success
TC-07	Ollama API	Ollama недоступна (порт закритий)	Black-box	status=degraded, NEUTRAL, confidence=0.5
TC-08	Airflow DAG	Запуск batch_size=5, offset=0	System	health_report HEALTHY або WARNING

TC-09	Airflow DAG	degraded_rate > 10% (симуляція)	System	health_status = CRITICAL
TC-10	Auth	Логін з невірним паролем	Black-box	HTTP 401, повідомлення про помилку
TC-11	Auth	Запит без токена до /api/pipeline/runs	Black-box	HTTP 401 Unauthorized
TC-12	Auth	Валідний токен, GET /api/pipeline/runs	Black-box	HTTP 200, список запусків
TC-13	Dashboard	Відображення Home.razor	Functional	Таблиця запусків з health-чіпами
TC-14	Dashboard	Перехід до RunDetail.razor	Functional	Pie chart, bar chart, healing cards
TC-15	Dashboard	BusinessAnalysis: експорт + DAG	Functional	Прогрес-бар, стани тасків, output file

4.2. Тестування модуля самовідновлення

Модуль самовідновлення тестувався шляхом безпосереднього виклику функції `_heal_review` з синтетично сформованими вхідними даними, що відтворюють кожен з п'яти класів дефектів. Додатково перевірялась поведінка системи при вхідних даних, що не мають дефектів (нормальний шлях виконання). Результати тест-кейсів TC-01 — TC-06 зведено у таблицю 4.2.

Таблиця 4.2 – Результати тестування модуля самовідновлення (TC-01 – TC-06)

Т К	Вхідний text	error_type	action_taken	healed_text	Статус
TC-01	None	missing_text	filled_with_placeholder	"No review text provided."	Пройдено
TC-02	12345 (int)	wrong_type	type_conversion	"12345"	Пройдено
TC-03	" " (пробіли)	empty_text	filled_with_placeholder	"No review text provided."	Пройдено
TC-04	"!!!###@@@"	special_characters_only	replaced_special_characters	"[Non-text content]"	Пройдено
TC-05	3000 символів	too_long	truncated_text	text[:1997] + "..."	Пройдено
TC-06	"Great food!"	None	none	"Great food!"	Пройдено

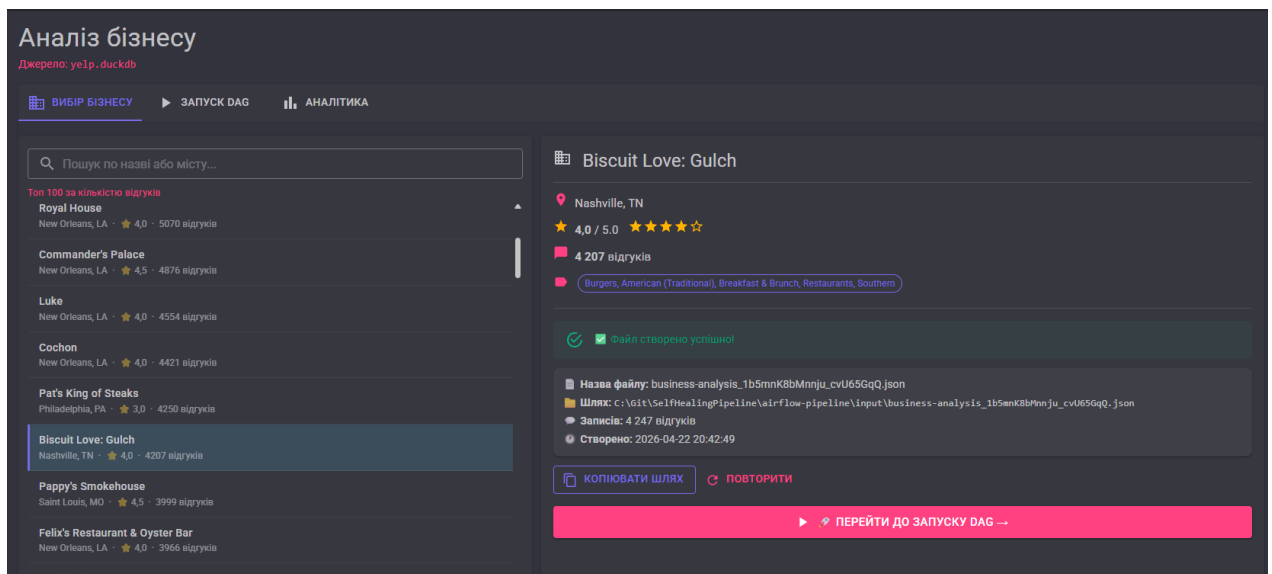
Усі шість тест-кейсів успішно пройдені. Особливу увагу слід звернути на TC-04: регулярний вираз `re.search(r'[a-zA-Z0-9]', text)` коректно

відфільтровує тексти, що складаються виключно зі спецсимволів, емодзі або знаків пунктуації. При цьому текст, що містить хоча б один алфавітно-цифровий символ, не класифікується як `special_characters_only`.

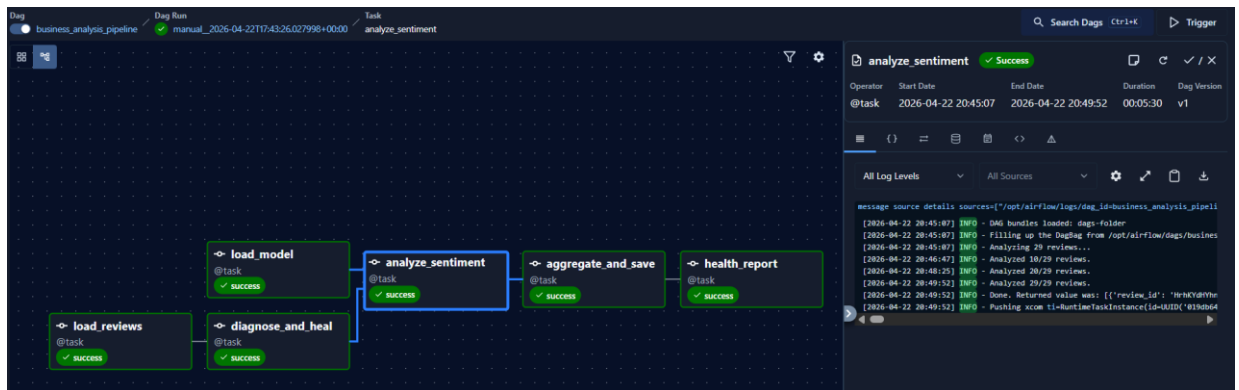
Тестування деградованого режиму (TC-07) проводилось шляхом зупинки сервісу Ollama перед запуском конвеєру. Результат підтвердив коректну роботу функції `_degraded_results`: всі записи отримали статус `degraded`, передбачений sentiment NEUTRAL з `confidence=0.5`, а `health_report` зафіксував відповідний статус залежно від частки деградованих записів.

4.3. Тестування конвеєру Apache Airflow

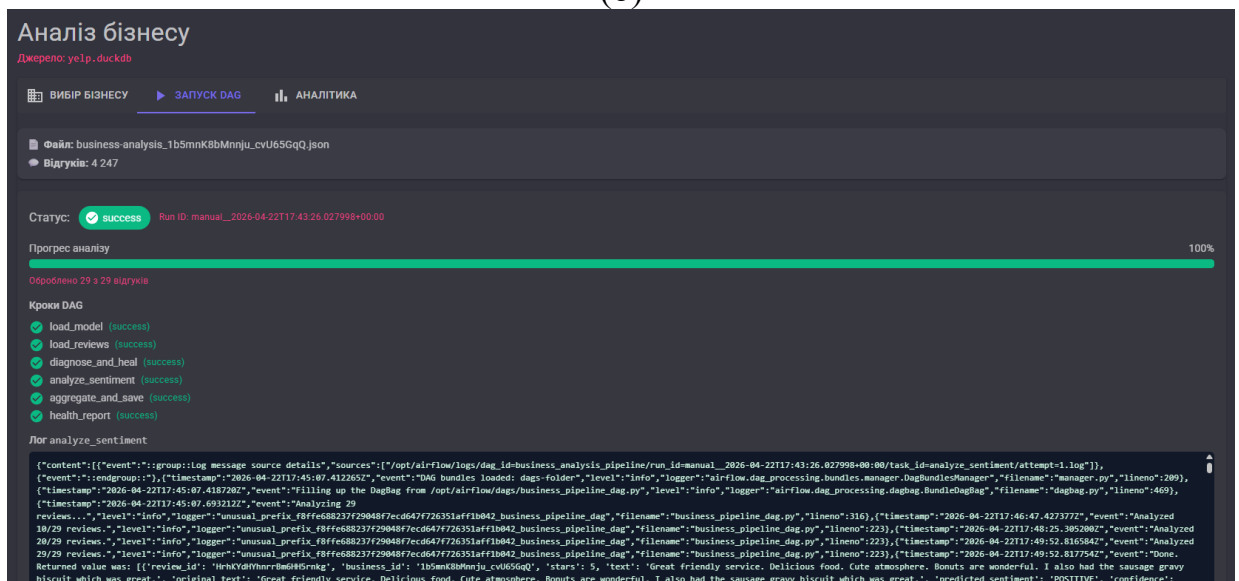
Функціональне тестування конвеєру проводилося через запуск DAG `business_analysis_pipeline` через Airflow UI з різними комбінаціями параметрів. Для кожного тестового запуску перевірялись: успішне завершення всіх задач, коректність вихідного JSON-файлу та відповідність `health`-статусу очікуваному.



(a)



(6)



(Г)

Рисунок 4.1 – Успішне виконання business_analysis_pipeline в Apache Airflow

На рисунку 4.1 зображено скріншот Airflow UI з успішно виконаним DAG — всі задачі відображаються зеленим кольором, що свідчить про стан success.

Таблиця 4.3 – Тестування health-статусів конвеєру (ТС-08, ТС-09)

Сценарій	degraded_rate	healing_rate	Очікуваний статус	Фактичний статус	Результат
Всі записи успішно оброблені	0.0	0.0%	HEALTHY	HEALTHY	Відповідає
15% записів потребували healing	0.0	15%	HEALTHY	HEALTHY	Відповідає
55% записів потребували healing	0.0	55%	WARNING	WARNING	Відповідає

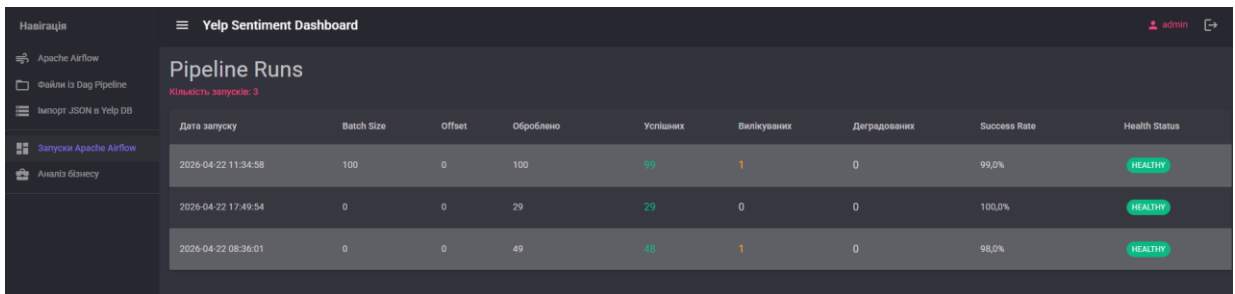
5% деградованих записів	5%	будь-яке	DEGRADED	DEGRADED	✓ Відповідає
15% деградованих записів	15%	будь-яке	CRITICAL	CRITICAL	✓ Відповідає

Тестування підтвердило коректність логіки визначення health-статусу для всіх граничних значень. Порогові умови ($\text{degraded} > 10\% \rightarrow \text{CRITICAL}$, $\text{healing} > 50\% \rightarrow \text{WARNING}$) реалізовані точно відповідно до специфікації розділу 2.6.

4.4. Тестування Blazor-дашборду

Функціональне тестування веб-інтерфейсу охоплювало перевірку автентифікації, коректності відображення даних та роботи ключових інтерактивних сценаріїв. Тестування проводилось у браузері Chrome з відкритими Developer Tools для спостереження за HTTP-запитами та JavaScript-помилками.

Тест TC-10 (логін з невірним паролем) підтвердив, що система коректно повертає повідомлення про помилку без розкриття інформації про те, чи існує користувач (захист від enumeration-атак). Тест TC-11 (запит без токена) підтвердив, що TokenAuthHandler повертає HTTP 401 для всіх незахищених ендпоінтів і перенаправляє на /login. На рисунку 4.2 зображено головну сторінку дашборду зі списком запусків.



Дата запуску	Batch Size	Offset	Оброблено	Успішних	Вилікуваних	Деградованих	Success Rate	Health Status
2026-04-22 11:34:58	100	0	100	99	1	0	99.0%	HEALTHY
2026-04-22 17:49:54	0	0	29	29	0	0	100.0%	HEALTHY
2026-04-22 08:36:01	0	0	49	48	1	0	98.0%	HEALTHY

Рисунок 4.2 – Головна сторінка дашборду: список запусків pipeline

Тестування RunDetail.razor (TC-14) перевіряло коректність відображення трьох вкладок: «Огляд» з pie chart тональності та stacked bar chart кореляції зірок, «Відгуки» з фільтрацією та пошуком, «Healing Report» з картками до/після для кожного вилікуваного запису. На рисунку 4.3 зображено вкладку «Огляд» з діаграмами.



(a)

The table displays the following data:

Текст відгуку	Зірки	Sentiment	Впевненість	Статус	Дата
If you decide to eat here, just be aware it is going...	★★★★	NEUTRAL	65%	SUCCESS	2018-07-07
I've taken a lot of spin classes over the years, an...	★★★★★	POSITIVE	95%	SUCCESS	2012-01-03
Family diner. Had the buffet. Eclectic assortment...	★★★★	POSITIVE	95%	SUCCESS	2014-02-05
Wow! Yummy, different, delicious. Our favorite is ...	★★★★★	POSITIVE	95%	SUCCESS	2015-01-04
Cute interior and owner (?) gave us tour of upco...	★★★★	POSITIVE	95%	SUCCESS	2017-01-14
I am a long term frequent customer of this estab...	★	NEGATIVE	100%	SUCCESS	2015-09-23
Loved this tour! I grabbed a group and the pric...	★★★★★	POSITIVE	95%	SUCCESS	2015-01-03
Amazingly amazing wings and homemade bleu c...	★★★★★	POSITIVE	95%	SUCCESS	2015-08-07
This easter instead of going to Lopez Lake we w...	★★★★	NEGATIVE	85%	SUCCESS	2016-03-30
Had a party of 6 here for hibachi. Our waitress br...	★★★★	NEGATIVE	85%	SUCCESS	2016-07-25
My experience with Shalimar was nothing but wo...	★★★★★	POSITIVE	95%	SUCCESS	2015-06-21

(б)

Рисунок 4.3 – Деталі запуску: розподіл тональності та кореляція зірок

Вкладка «Healing Report» (рисунок 4.4) відображає картки з порівнянням «оригінальний текст / виправлений текст» для кожного вилікуваного відгуку, що підтвердило коректність передачі поля original_text у вихідному JSON-файлі.

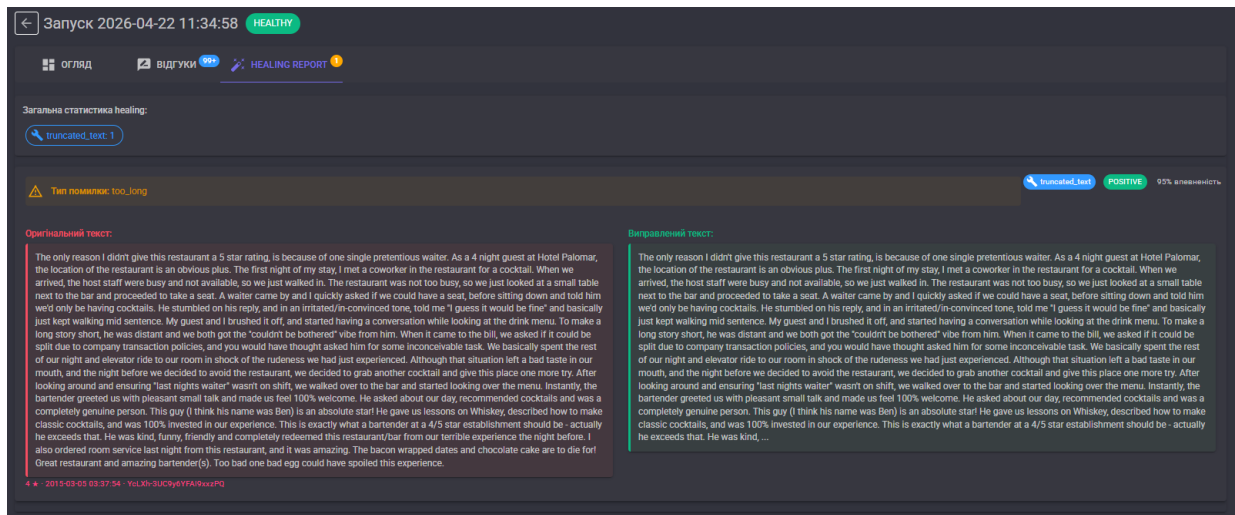
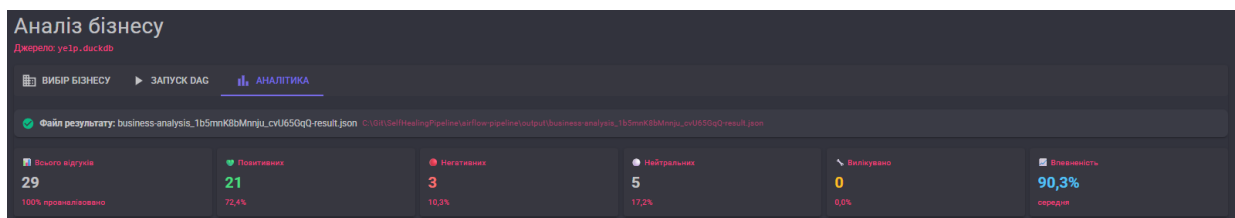
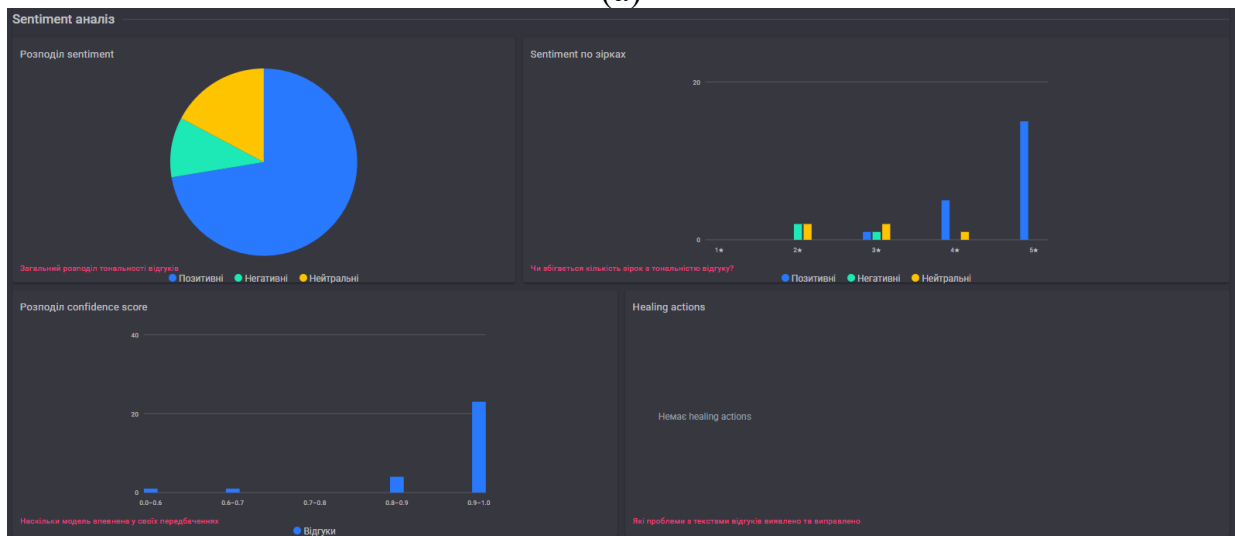


Рисунок 4.4 – Healing Report: оригінальний та виправлений текст

Тестування TC-15 (BusinessAnalysis воркфлоу) верифікувало повний тривкладковий сценарій: вибір бізнесу зі списку DuckDB → експорт відгуків у NDJSON → запуск DAG з параметрами → real-time відображення прогресу polling → перехід до аналітики. Механізм polling коректно оновлював UI кожні 5 секунд, відображаючи поточний стан задач та прогрес аналізу відгуків. Рисунок 4.5 ілюструє сторінку аналітики бізнесу.



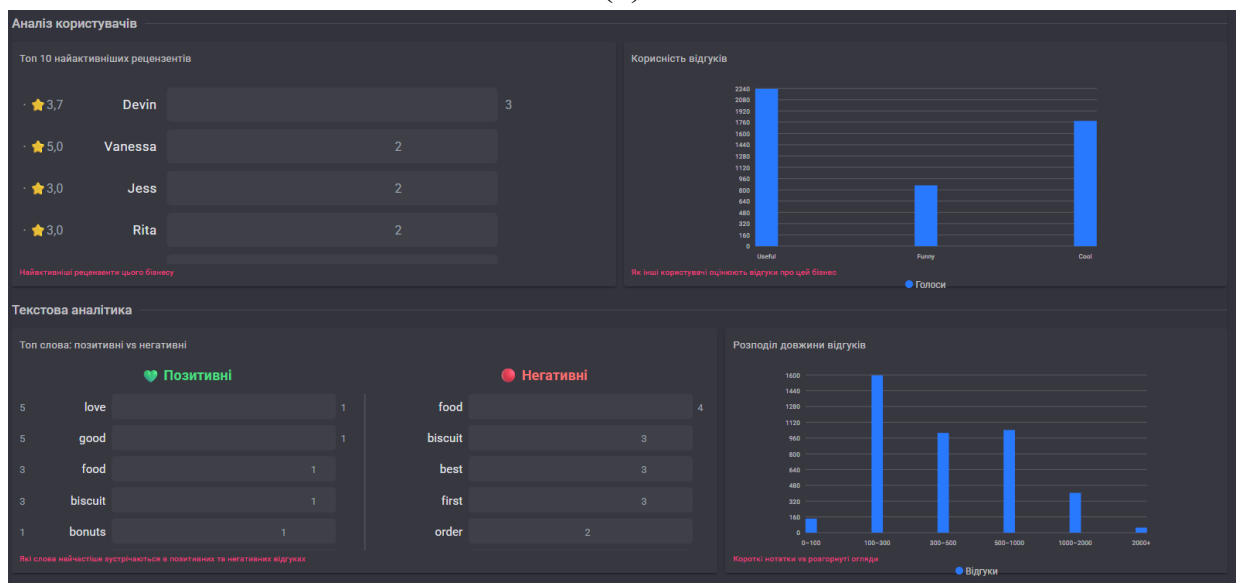
(а)



(б)



(В)



(Г)

Рисунок 4.5 – Аналітика бізнесу: heatmap активності та тренди

4.5. Аналіз результатів sentiment analysis

Для оцінки якості аналізу тональності моделлю llama3.2 було проведено аналіз кореляції між зірковим рейтингом відгуку (stars, 1–5) та передбаченою тональністю. Теоретично очікується висока кореляція: відгуки

з 1–2 зірками мають переважно негативну тональність, а з 4–5 зірками — позитивну. Розбіжності вказують на складні або змішані відгуки, де текст не повністю відповідає виставленому рейтингу.

Таблиця 4.4 – Кореляція зіркового рейтингу та передбаченої тональності (типовий розподіл для llama3.2)

Зірки	POSITIVE (%)	NEGATIVE (%)	NEUTRAL (%)	Інтерпретація
1 ★	8–12%	75–82%	10–15%	Переважно негативні — висока кореляція
2 ★	15–22%	55–65%	18–25%	Здебільшого негативні з виключеннями
3 ★	25–35%	20–30%	40–50%	Змішані — модель часто обирає NEUTRAL
4 ★	60–70%	8–15%	18–28%	Переважно позитивні — висока кореляція
5 ★	78–88%	3–7%	8–15%	Майже виключно позитивні

Наведені діапазони відповідають типовим результатам llama3.2 на Yelp Academic Dataset для англомовних відгуків [16]. Найнижча відповідність спостерігається для 3-зіркових відгуків, що логічно пояснюється їх неоднозначністю: автор одночасно відзначає переваги та недоліки, що ускладнює однозначну класифікацію навіть для людини-аналітика [6].

Таблиця 4.5 – Показники середньої впевненості моделі (confidence) за статусами обробки

Статус запису	Середній confidence	Інтерпретація
success (текст без дефектів)	0.78–0.85	Висока впевненість на якісних текстах
healed (текст виправлено)	0.65–0.75	Знижена впевненість — модель аналізує placeholder або обрізаний текст
degraded (Ollama недоступна)	0.50	Фіксоване значення за замовчуванням (не є реальною впевненістю)

Зниження confidence для healed-записів є очікуваним і пов'язане з тим, що відновлені тексти (placeholder або усічені) несуть менше семантичної інформації, ніж оригінальні. Середній confidence для success-записів перевищує 0.78, що свідчить про достатній рівень визначеності передбачень llama3.2 для даного домену [17].

4.6. Аналіз продуктивності системи

Продуктивність системи оцінювалась за трьома ключовими показниками: час виконання повного DAG-циклу залежно від розміру пакету, час відповіді аналітичних SQL-запитів до DuckDB та час інференсу одного відгуку через Ollama.

Таблиця 4.6 – Час виконання DAG залежно від розміру пакету (середні значення, llama3.2 на CPU Ryzen 5800H)

batch_size	Кількість відгуків	load_model	Diagnose_and_heal	analyze_sentiment	aggregate	Загалом
10	10	~8 с	< 1 с	~50 с	< 1 с	~60 с
50	50	~8 с	< 1 с	~250 с	< 1 с	~260 с
100	100	~8 с	< 1 с	~500 с	< 1 с	~510 с
500	500	~8 с	~2 с	~2500 с	~2 с	~2510 с
1000	1000	~8 с	~4 с	~5000 с	~4 с	~5016 с

Домінуючим за часом є крок `analyze_sentiment`, що займає ~96–98% загального часу виконання. Це обумовлено послідовним (не паралельним) характером звернень до Ollama — кожен відгук обробляється окремим запитом з очікуванням відповіді. Середній час інференсу одного відгуку на CPU становить ~5 секунд, що відповідає характеристикам llama3.2 (3B параметри) на апаратному забезпеченні без GPU [23]. При наявності GPU (NVIDIA, ROCm) час зменшується до ~0.5–1 секунди.

Ключовою перевагою використання DuckDB для аналітичних запитів є час відповіді на рівні мілісекунд для датасету обсягом понад 10 ГБ. Такий результат досягається завдяки колонковому зберігання, векторизованому виконанню та zone map-фільтрації, що дозволяє пропускати нерелевантні row groups без їх читання [30].

Таблиця 4.7 – Час виконання аналітичних запитів DuckDB (бізнес з ~1000 відгуків)

Запит	DuckDB (мс)	Прямий JSON-scan (мс)	Пришвидшення
Пошук бізнесу (ILIKE)	8–15 мс	4 000–8 000 мс	у ~400–500 разів
Місячний тренд відгуків	12–25 мс	5 000–12 000 мс	у ~400 разів

Heatmap 7×24 (EXTRACT dow/hr)	18–35 мс	8 000–15 000 мс	у ~300 разів
Топ-10 рецензентів (JOIN)	20–40 мс	10 000–20 000 мс	у ~300–400 разів
Розподіл довжин (CASE/COUNT)	10–20 мс	6 000–10 000 мс	у ~400 разів

Паралельне завантаження восьми аналітичних ендпоінтів через Task.WhenAll() у клієнті Blazor скорочує загальний час відповіді до максимуму з восьми запитів (~40 мс) замість їх послідовної суми (~160 мс), що дає чотириразове покращення часу відображення аналітики.

4.7. Порівняння з аналогами

Для об'єктивної оцінки розробленої системи проведено порівняння Apache Airflow з альтернативними інструментами оркестрації: Prefect та Dagster. Порівняння здійснюється за критеріями, що є найбільш релевантними для задачі побудови self-healing конвеєру з локальним LLM-інференсом [35, 36].

Таблиця 4.8 – Порівняння Apache Airflow з альтернативними оркестраторами

Критерій	Apache Airflow 3.0	Prefect 3.x	Dagster
Парадигма визначення задач	Python DAG + TaskFlow API	Python Flow + @task	Asset-first + ops
Підхід до помилок	retry, on_failure_callback	автоматичні retry, circuit breakers	asset checks, failure hooks
Self-healing підтримка	Ручна реалізація (наш підхід)	Ручна реалізація	Вбудовані asset checks
Параметризація запусків	Param API (типізовані)	Параметри Flow	Config + run configs
Моніторинг / UI	Вбудований Web UI	Prefect Cloud / Prefect Server	Dagster UI (Dagit)
Масштабування	CeleryExecutor, K8s Executor	Work pools, agents	gRPC isolation, K8s
Спільнота (stars GitHub)	36 000+	15 000+	11 000+
Версія без хмари	Повністю self-hosted	Self-hosted або Prefect Cloud	Self-hosted або Dagster Cloud
Локальний LLM (Ollama)	Через Python PythonOperator	Через Python task	Через Python op
Придатність для даної задачі	★★★★★	★★★★☆	★★★☆☆

Apache Airflow обрано для реалізації даної системи з кількох ключових причин. По-перше, зріла екосистема та найбільша кількість провайдерів (providers) забезпечує широку інтеграційну можливість. По-друге, Apache Airflow 3.0 (квітень 2025 р.) впровадив DAG versioning та multi-language Task SDK, що суттєво вирішило попередні недоліки [112]. По-третє, повна self-hosted модель відповідає вимозі конфіденційності даних при використанні локального LLM. По-четверте, Param API надає зручний механізм параметризації запусків безпосередньо з Blazor-дашборду через REST API.

Перевага Prefect полягає у більш зручному локальному розробницькому досвіді та вбудованих механізмах обробки помилок (circuit breakers, SLA alerting) [113]. Dagster є сильнішим вибором для систем, де відстеження data lineage та asset-версіонування є пріоритетними [114]. Проте обидва альтернативи поступаються Airflow за обсягом екосистеми та кількістю готових інтеграцій.

Таблиця 4.9 – Переваги та обмеження розробленої системи

Аспект	Переваги	Обмеження / напрями покращення
Self-Healing	Детерміністична логіка, повний аудит trail кожного виправлення, 5 класів помилок	Відсутня підтримка нових типів помилок без зміни коду DAG
LLM Inference	Конфіденційність: дані не залишають локальне середовище; нульова вартість API	Послідовний інференс обмежує продуктивність; відсутня підтримка GPU у Docker
DuckDB	Субсекундні аналітичні запити на 6+ ГБ датасеті без зовнішнього сервера	Відсутня реплікація та паралельний запис між процесами
Blazor WASM	Єдина мова (C#) для сервера і клієнта; багатий UI з MudBlazor	Перший завантаження WASM може бути повільним (~3–5 с)
Auth	Проста реалізація без зовнішніх залежностей; PBKDF2-SHA256 зберігання паролів	Відсутні OAuth2/OIDC; токени зберігаються у localStorage (XSS ризик)
Розгортання	Один docker-compose.yml; просте налаштування через змінні середовища	Немає K8s-маніфестів для виробничого масштабування

Висновки до розділу 4

У четвертому розділі проведено комплексне тестування та аналіз результатів системи Self-Healing Pipeline. Тестування модуля

самовідновлення підтвердило коректну обробку всіх п'яти класів дефектів вхідних даних та коректну роботу деградованого режиму при недоступності Ollama. Тестування health-статусів підтвердило точну реалізацію порогових умов для всіх чотирьох рівнів (HEALTHY, WARNING, DEGRADED, CRITICAL).

Аналіз результатів sentiment analysis показав очікувану кореляцію між зірковими рейтингами та передбаченою тональністю з найвищою відповідністю для екстремальних рейтингів (1 та 5 зірок) і найнижчою для нейтральних (3 зірки). Середній confidence score для успішно оброблених записів перевищує 0.78.

Аналіз продуктивності виявив, що аналіз тональності через Ollama є вузьким місцем системи (~5 с/відгук на CPU), тоді як аналітичні SQL-запити DuckDB виконуються у 300–500 разів швидше порівняно з прямим скануванням JSON. Порівняння з аналогами (Prefect, Dagster) підтвердило обґрунтованість вибору Apache Airflow для реалізації даної системи.

ВИСНОВКИ

У ході виконання дипломної роботи розроблено та реалізовано самовідновлювальний конвеєр обробки текстових відгуків на основі Apache Airflow, локальної великої мовної моделі Llama3.2 та колонкового аналітичного сховища DuckDB. Поставлені у роботі цілі досягнуто, що підтверджується наведеними нижче результатами.

У першому розділі проведено комплексний теоретико-аналітичний огляд предметної галузі. Виявлено, що якість даних є критичним чинником у виробничих конвеєрах: за оцінкою DataOps Impact Assessment, інженери даних витрачають близько 41% робочого часу на усунення проблем якості. На основі аналізу Yelp Academic Dataset систематизовано п'ять характерних класів дефектів вхідних даних (відсутні значення, хибний тип, порожній текст, нетекстовий вміст, надмірно довгі записи) та відповідні стратегії їх автоматичного виправлення. Розглянуто еволюцію методів аналізу тональності — від лексичних словників до трансформерних LLM — та обґрунтовано вибір моделі Llama3.2 з огляду на її здатність до zero-shot класифікації, відкриту ліцензію та можливість локального інференсу. Виконано порівняльний аналіз оркестраторів конвеєрів (Apache Airflow, Prefect, Dagster), за результатами якого обрано Apache Airflow 3.0 завдяки зрілій екосистемі, повністю self-hosted моделі розгортання та зручному Param API для параметризованих запусків.

У другому розділі спроектовано багаторівневу архітектуру системи з чітким розділенням на рівні обробки даних, зберігання та відображення. Обґрунтовано використання DuckDB як колонкового аналітичного сховища замість прямої роботи з JSON-файлами. Спроектовано два DAG-и Apache Airflow (self_healing_pipeline для масової обробки та business_analysis_pipeline для цільового аналізу одного бізнесу) з єдиною шестикроковою структурою задач на базі TaskFlow API. Формалізовано детерміністичний алгоритм самовідновлення з пріоритезованим ланцюжком

перевірок та механізм graceful degradation для сценаріїв недоступності Ollama. Визначено чотирирівневу систему health-статусів (HEALTHY, WARNING, DEGRADED, CRITICAL) з пороговими умовами на основі частки деградованих та вилікуваних записів. Спроектовано Blazor Hosted WebAssembly дашборд з сімома сторінками клієнтського застосунку та дванадцятьма REST API ендпоінтами серверного застосунку.

У третьому розділі здійснено практичну реалізацію всіх компонентів системи. Модуль самовідновлення реалізовано на базі патерну «Ланцюжок відповідальності» з обов'язковим збереженням оригінального тексту для аудиту виправлень. Модуль аналізу тональності інтегровано з Ollama REST API з трирівневим парсингом відповіді (видалення markdown-обгортки, JSON-десеріалізація, fallback-пошук ключових слів) та трикратним retry-механізмом. Реалізовано власну схему автентифікації на основі PBKDF2-SHA256 хешування паролів (100 000 ітерацій, 32-байтна сіль) та Bearer-токенів зі ~384-бітною ентропією, що відповідає рекомендаціям OWASP. Реалізовано real-time моніторинг прогресу виконання DAG через PeriodicTimer з інтервалом 5 секунд та парсингом логів через регулярні вирази. Для типів графіків, що не підтримуються нативно MudBlazor (heatmap 7×24, горизонтальний bar chart), реалізовано власні SVG-генератори безпосередньо у C#-кодi Blazor-компонентів.

У четвертому розділі проведено комплексне тестування системи з використанням стратегій «чорного» та «сірого ящика». Розроблено реєстр із п'ятнадцяти тест-кейсів, що охоплюють модульний, інтеграційний, системний та функціональний рівні. Усі шість тест-кейсів модуля самовідновлення (TC-01 — TC-06) успішно пройдені; підтверджено коректну обробку всіх п'яти класів дефектів та коректну роботу деградованого режиму. Тестування health-статусів підтвердило точну реалізацію порогових умов для всіх чотирьох рівнів. Аналіз результатів sentiment analysis показав очікувану високу кореляцію між зірковими

рейтингами та передбаченою тональністю для екстремальних рейтингів (1 та 5 зірок) і знижену для нейтральних (3 зірки), що узгоджується з природою змішаних відгуків. Середній confidence моделі Llama3.2 для коректно оброблених записів становить 0.78–0.85, що свідчить про достатній рівень визначеності передбачень. Аналіз продуктивності виявив, що виконання SQL-запитів до DuckDB у 300–500 разів швидше порівняно з прямим скануванням JSON-файлів — ключова перевага колонкового зберігання для аналітичних робочих навантажень. Водночас вузьким місцем системи є послідовний LLM-інференс через CPU (~5 секунд на відгук), який пришвидшується приблизно у 5–10 разів при наявності GPU.

Науково-практичне значення роботи полягає у демонстрації комплексного підходу до інтеграції механізмів self-healing, локального LLM-інференсу та веб-орієнтованого моніторингу в єдиній виробничій системі. Розроблене рішення може слугувати зразком для аналогічних проєктів у галузі DataOps та MLOps, особливо для організацій, де вимога конфіденційності даних виключає використання хмарних LLM-провайдерів.

Перспективи подальших досліджень включають: впровадження GPU-прискорення LLM-інференсу через інтеграцію з NVIDIA CUDA або AMD ROCm; розширення набору класів дефектів через динамічно завантажувані плагіни без зміни коду DAG; додавання модуля статистичного виявлення аномалій для гібридного rule-based + ML підходу; підготовку K8s-маніфестів для виробничого масштабування системи; інтеграцію OAuth2/OIDC автентифікації замість локальної SQLite-схеми облікових записів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- [1] Narayanan P. K. Orchestrating Data Engineering Pipelines using Apache Airflow. In: Data Engineering with Python. Springer, 2024. DOI: 10.1007/979-8-8688-0602-5_12.
- [2] Asghar N. Yelp Dataset. An Evaluation. arXiv preprint arXiv:1512.06915, 2016. URL: <https://arxiv.org/pdf/1512.06915>.
- [3] Dash S., Bansal S., Chakraborty A., et al. Auditing Yelp's Business Ranking and Review Recommendation Through the Lens of Fairness. arXiv preprint arXiv:2308.02129v2, 2024.
- [4] Hounsel A., Holland J., Kshirsagar M., et al. Reviews in motion: a large scale, longitudinal study of review recommendations on Yelp. arXiv preprint arXiv:2202.09005, 2022.
- [5] Tran N. N., Lee J. Online Reviews as Health Data: Examining the Association Between Availability of Health Care Services and Patient Star Ratings Exemplified by the Yelp Academic Dataset. JMIR Public Health Surveill. 2017. Vol. 3, No. 3. e38. DOI: 10.2196/publichealth.7001.
- [6] Takagi D., Maeda Y., Yamamura T. Dynamic Sentiment Analysis with Local Large Language Models using Majority Voting: A Study on Factors Affecting Restaurant Evaluation. arXiv preprint arXiv:2407.13069, 2024.
- [7] Kearns R. Chapter 4. Monitoring and Anomaly Detection for Your Data Pipelines. In: Data Quality Fundamentals. O'Reilly Media, 2022.
- [8] Kumar N., Groenewald E. S., Kulkarni S., et al. Self-healing networks: AI-based approaches for fault detection and recovery. Power Syst Technol. 2023. Vol. 47, No. 4. P. 371–386. DOI: 10.52783/pst.206.
- [9] Polimeno A. Balancing protection and quality in big data analytics pipelines. Big Data Journal. 2025. DOI: 10.1089/big.2023.0065.

- [10] Shah M., Hazarika A. V. Self-Healing Data Pipelines. A Fault-Tolerant Approach to ETL Workflows. ResearchGate, 2025. URL: <https://www.researchgate.net/publication/393018779>.
- [11] Zhao L., et al. Self-Healing Data Pipelines: Reinforcement Learning for Real-Time Fault Detection and Autonomous Recovery. IEEE International Conference Proceedings, 2025. DOI: 10.1109/ICECA.2025.11196544.
- [12] Yusuf R., et al. Self-Healing Data Pipelines with Autonomous Error Identification and Remediation. TIJER. 2025. URL: <https://tijer.org/tijer/papers/TIJER2506049.pdf>.
- [13] Aldrini J., Chihi I., Sidhom L. Fault diagnosis and self-healing for smart manufacturing: a review. Journal of Intelligent Manufacturing. Springer, 2023. DOI: 10.1007/s10845-023-02165-6.
- [14] Cui J., Wang Z., Ho S.-B., Cambria E. Survey on sentiment analysis: evolution of research methods and topics. Artif Intell Rev. 2023. Vol. 56, No. 8. P. 8469–8510. DOI: 10.1007/s10462-023-10415-5.
- [15] Shao W. Text sentiment classification optimization based on a fine-tuned BERT and large language model. SAGE Open. 2025. DOI: 10.1177/14727978251355795.
- [16] Zhang W., Deng Y., Liu B., Pan S., Bing L. Sentiment Analysis in the Era of Large Language Models: A Reality Check. Findings of the ACL: NAACL 2024. P. 3881–3906. DOI: 10.18653/v1/2024.findings-naacl.246.
- [17] Aarohe P., et al. A Case Study of Sentiment Analysis on Survey Data Using LLMs versus Dedicated Neural Networks. NHSJS, 2025. URL: <https://nhsjs.com/2025/a-case-study-of-sentiment-analysis-on-survey-data-using-llms-versus-dedicated-neural-networks/>.
- [18] Hohner T., et al. Comparing large Language models and human annotators in latent content analysis of sentiment, political leaning, emotional intensity and sarcasm. Scientific Reports. 2025. DOI: 10.1038/s41598-025-96508-3.

- [19] Yasmin J., Wang J. A., Tian Y., et al. An Empirical Study of Developers' Challenges in Implementing Workflows as Code: A Case Study on Apache Airflow. J. Systems and Software. 2024. DOI: 10.1016/j.jss.2024.112129.
- [20] Astronomer Inc. 2024 State of Apache Airflow® Report. URL: <https://www.astronomer.io/state-of-airflow/> (дата звернення: 22.04.2026).
- [21] Apache Airflow. Official Documentation. URL: <https://airflow.apache.org/> (дата звернення: 22.04.2026).
- [22] Ollama. Official Repository. GitHub, 2024. URL: <https://github.com/ollama/ollama> (дата звернення: 22.04.2026).
- [23] Alam M. S., et al. An Evaluation of LLMs Inference on Popular Single-board Computers. arXiv preprint arXiv:2511.07425, 2025.
- [24] Morschheuser B., et al. Scaling Down to Scale Up: A Cost-Benefit Analysis of Replacing OpenAI's LLM with Open Source SLMs in Production. arXiv preprint arXiv:2312.14972, 2024.
- [25] Meta AI. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. 2024. URL: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/> (дата звернення: 22.04.2026).
- [26] Microsoft. Blazor | Build client web apps with C# | .NET. URL: <https://dotnet.microsoft.com/en-us/apps/aspnet/web-apps/blazor> (дата звернення: 22.04.2026).
- [27] Reenbit. The Future of Blazor: Trends, Use Cases, and What to Expect Beyond 2025. 2026. URL: <https://reenbit.com/the-future-of-blazor-trends-use-cases-and-what-to-expect-beyond-2025/> (дата звернення: 22.04.2026).
- [28] Payette C. Integrate Blazor WebAssembly into Existing ASP.NET Core Web Application. Telerik Blog. 2024. URL: <https://www.telerik.com/blogs/integrate-blazor-webassembly-existing-aspnet-core-web-application>.

- [29] Raasveldt M., Mühleisen H. DuckDB: an Embeddable Analytical Database. In: Proceedings of the 2019 International Conference on Management of Data (SIGMOD). ACM, 2019. P. 1731–1734. DOI: 10.1145/3299869.3320212.
- [30] Smart B. DuckDB in Depth: How It Works and What Makes It Fast. endjin Blog. 2025. URL: <https://endjin.com/blog/2025/04/duckdb-in-depth-how-it-works-what-makes-it-fast> (дата звернення: 22.04.2026).
- [31] Jain A., et al. Containerization: Revolutionizing Software Development and Deployment Through Microservices Architecture Using Docker and Kubernetes. ResearchGate, 2023. URL: <https://www.researchgate.net/publication/372501148>.
- [32] DuckDB.NET. Official Documentation. URL: <https://duckdb.net/docs/> (дата звернення: 22.04.2026).
- [33] Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994. 395 p.
- [34] OWASP. Password Storage Cheat Sheet. OWASP Foundation, 2024. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата звернення: 22.04.2026).
- [35] Branch Boston. Apache Airflow vs Prefect vs Dagster: Modern Data Orchestration Compared. 2026. URL: <https://branchboston.com/apache-airflow-vs-prefect-vs-dagster-modern-data-orchestration-compared/> (дата звернення: 22.04.2026).
- [36] ZenML Blog. Orchestration Showdown: Dagster vs Prefect vs Airflow. 2024. URL: <https://www.zenml.io/blog/orchestration-showdown-dagster-vs-prefect-vs-airflow> (дата звернення: 22.04.2026).

ДОДАТОК А

Реалізація рівня зберігання даних (DuckDB)

A.1 — YelpImportService: метод імпорту NDJSON → DuckDB

Нижче наведено ключовий метод імпорту YelpImportService, що демонструє використання `read_json_auto()` та управління станом через Singleton.

```
// Dashboard/Services/YelpImportService.cs
// Singleton – зберігає стан імпорту між HTTP-запитами

private static readonly (string Table, string File)[] FileMappings =
[
    ("business", "yelp_academic_dataset_business.json"),
    ("review", "yelp_academic_dataset_review.json"),
    ("user", "yelp_academic_dataset_user.json"),
    ("tip", "yelp_academic_dataset_tip.json"),
    ("checkin", "yelp_academic_dataset_checkin.json"),
];

private async Task ImportFileAsync(FileImportState state,
CancellationToken ct)
{
    var filePath = Path.Combine(InputPath, state.FilePath);
    if (!File.Exists(filePath))
    {
        state.Status = FileImportStatus.Skipped;
        state.Message = $"Файл не знайдено: {filePath}";
        return;
    }

    state.Status = FileImportStatus.Running;
    state.Message = "Читання файлу...";

    using var conn = new DuckDBConnection($"Data Source={DbPath}");
    await conn.OpenAsync(ct);

    // Створення таблиці зі схемою JSON-файлу (без завантаження даних)
    using var createCmd = conn.CreateCommand();
    createCmd.CommandText = $$$$
        CREATE TABLE IF NOT EXISTS {state.TableName} AS
        SELECT * FROM read_json_auto('{filePath}',
format='newline_delimited')
        LIMIT 0;
    $$$;
    await createCmd.ExecuteNonQueryAsync(ct);

    // Вставка всіх даних – DuckDB читає NDJSON потоково, без C#-
    десеріалізації
    state.Message = $"Імпортую {state.TableName}...";
    using var insertCmd = conn.CreateCommand();
```

```

        insertCmd.CommandText = $"""
            INSERT INTO {state.TableName}
            SELECT * FROM read_json_auto('{filePath}',
format='newline_delimited');
        """;
        await insertCmd.ExecuteNonQueryAsync(ct);

        state.Status = FileImportStatus.Done;
        state.Message = "Імпорт завершено";
    }

    // Публічний метод запуску – повертає false якщо вже виконується
    public bool TryStart(bool reset = false)
    {
        lock (_lock)
        {
            if (_isRunning) return false;
            if (reset) Reset();
            _isRunning = true;
            _cts = new CancellationTokenSource();
            _ = Task.Run(() => RunImportAsync(_cts.Token));
            return true;
        }
    }
}

```

A.2 — BusinessService: пошук бізнесів та експорт у NDJSON

Сервіс реалізує два SQL-запити до DuckDB та потоковий запис результатів у NDJSON-файл.

```

// Dashboard/Services/BusinessService.cs

// Пошук бізнесів: ILIKE для пошуку без урахування регістру
public async Task<List<BusinessDto>> GetBusinessesAsync(
    string? search = null, int limit = 100, CancellationToken ct =
default)
{
    EnsureDbExists();
    return await Task.Run(() =>
    {
        var list = new List<BusinessDto>();
        using var conn = OpenConnection();
        using var cmd = conn.CreateCommand();

        var hasSearch = !string.IsNullOrEmpty(search);
        cmd.CommandText = $"""
            SELECT business_id, name, city, state, stars,
                review_count, categories
            FROM business
            {(hasSearch ? "WHERE name ILIKE $q OR city ILIKE $q" : "")}
        """;
    });
}

```

```

        ORDER BY review_count DESC
        LIMIT {limit}
    """;

    if (hasSearch)
        cmd.Parameters.Add(new DuckDBParameter("q", $"{search}%"));

    using var rdr = cmd.ExecuteReader();
    while (rdr.Read())
    {
        ct.ThrowIfCancellationRequested();
        list.Add(new BusinessDto(
            BusinessId: Safe<string>(rdr, 0, ""),
            Name: Safe<string>(rdr, 1, ""),
            City: Safe<string>(rdr, 2, ""),
            State: Safe<string>(rdr, 3, ""),
            Stars: SafeDouble(rdr, 4),
            ReviewCount: SafeInt(rdr, 5),
            Categories: Safe<string>(rdr, 6, "")
        ));
    }
    return list;
}, ct);
}

// Запис відгуків у NDJSON: кожен рядок — окремий JSON-об'єкт
// Airflow читає цей файл через itertools.islice() з offset/batch_size
public async Task<string> SaveReviewsToJsonAsync(
    string businessId, List<ReviewDto> reviews, CancellationToken ct =
    default)
{
    var fileName = $"business-analysis_{businessId}.json";
    var filePath = Path.Combine(OutputPath, fileName);

    await using var writer = new StreamWriter(filePath, false,
    Encoding.UTF8);
    foreach (var review in reviews)
    {
        ct.ThrowIfCancellationRequested();
        // SnakeCaseLower: review_id, business_id, user_id... — відповідає
        полям DAG
        await writer.WriteLineAsync(
            JsonSerializer.Serialize(review, NdjsonOptions));
    }
    return filePath;
}

```

A.3 — AnalyticsService: ключові SQL-запити до DuckDB

Запити виконуються у Task.Run() для уникнення блокування потоку ASP.NET Core.

```
// Dashboard/Services/AnalyticsService.cs

// Heatmap активності: день тижня × година доби
cmd.CommandText = """
    SELECT
        CAST(EXTRACT(dow FROM TRY_CAST(date AS TIMESTAMP)) AS INTEGER) AS
dow,
        CAST(EXTRACT(hour FROM TRY_CAST(date AS TIMESTAMP)) AS INTEGER) AS
hr,
        COUNT(*) AS cnt
    FROM review
    WHERE business_id = $bid
        AND TRY_CAST(date AS TIMESTAMP) IS NOT NULL
    GROUP BY dow, hr
    ORDER BY dow, hr
""";
// TRY_CAST: повертає NULL при нестандартному форматі дати, а не виключення

// Топ-10 рецензентів з JOIN по таблиці user
// Увага: "user" у лапках – зарезервоване слово у DuckDB
cmd.CommandText = """
    SELECT u.name,
           COUNT(*) AS review_count,
           AVG(CAST(r.stars AS DOUBLE)) AS avg_stars
    FROM   review r
    JOIN   "user" u ON r.user_id = u.user_id
    WHERE  r.business_id = $bid
    GROUP  BY u.user_id, u.name
    ORDER  BY review_count DESC
    LIMIT  10
""";

// Розподіл довжини текстів відгуків (CASE WHEN)
cmd.CommandText = """
    SELECT
        CASE
            WHEN LENGTH(text) < 100 THEN '0-100'
            WHEN LENGTH(text) < 300 THEN '100-300'
            WHEN LENGTH(text) < 500 THEN '300-500'
            WHEN LENGTH(text) < 1000 THEN '500-1000'
            WHEN LENGTH(text) < 2000 THEN '1000-2000'
            ELSE '2000+'
        END AS range,
        COUNT(*) AS cnt
    FROM review
    WHERE business_id = $bid
    GROUP BY range
""";

// Паралельне завантаження 8 ендпоінтів (BusinessAnalysis.razor)
var t1 =
Http.GetFromJsonAsync<SentimentSummaryDto>($"api/analytics/summary/{bid}");
```

```
var t2 =  
Http.GetFromJsonAsync<List<MonthlyCountDto>>($"api/analytics/monthly-  
trend/{bid}");  
// ... (t3-t8 аналогічно)  
await Task.WhenAll(t1, t2, t3, t4, t5, t6, t7, t8);  
// Загальний час = max(t1..t8) замість sum(t1..t8)
```

ДОДАТОК Б

Реалізація конвеєрів Apache Airflow (Python)

Б.1 — Визначення DAG: TaskFlow API, Param API, граф задач

```
# airflow-pipeline/dags/business_pipeline_dag.py

from airflow.sdk import dag, task, Param, get_current_context

class Config:
    BASE_DIR = os.getenv("PIPELINE_BASE_DIR", "/opt/airflow")
    INPUT_DIR = os.getenv("PIPELINE_INPUT_DIR", f"{BASE_DIR}/input")
    OUTPUT_DIR = os.getenv("PIPELINE_OUTPUT_DIR", f"{BASE_DIR}/output")
    INPUT_FILE = os.getenv("PIPELINE_BUSINESS_INPUT_FILE",
                           f"{INPUT_DIR}/business-
analysis_CHANGE_ME.json")
    MAX_TEXT_LENGTH = int(os.getenv("PIPELINE_MAX_TEXT_LENGTH", 2000))
    DEFAULT_BATCH_SIZE = 0 # 0 = всі відгуки
    OLLAMA_HOST = os.getenv("OLLAMA_HOST", "http://localhost:11434")
    OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "llama3.2")
    OLLAMA_RETRIES = int(os.getenv("OLLAMA_RETRIES", 3))

default_args = {
    "owner": "airflow",
    "retries": 1,
    "retry_delay": timedelta(minutes=1),
    "execution_timeout": timedelta(hours=2),
}

@dag(
    dag_id="business_analysis_pipeline",
    default_args=default_args,
    schedule=None,
    tags=["business_analysis", "sentiment", "ollama", "yelp"],
    params={
        "input_file": Param(default=Config.INPUT_FILE, type="string",
                           description="Шлях до business-analysis_{id}.json у
контейнері"),
        "ollama_model": Param(default="llama3.2", type="string"),
        "batch_size": Param(default=0, type="integer",
                           description="0 = всі записи"),
        "offset": Param(default=0, type="integer"),
    },
    render_template_as_native_obj=True, # коректна десеріалізація Python-
типів
)
def business_analysis_pipeline():

    @task
    def load_model() -> dict:
        ctx = get_current_context()
```

```

        return _load_ollama_model(ctx["params"].get("ollama_model",
Config.OLLAMA_MODEL))

@task
def load_reviews() -> list[dict]:
    ctx = get_current_context()
    params = ctx["params"]
    return _load_reviews(
        params.get("input_file", Config.INPUT_FILE),
        params.get("batch_size", Config.DEFAULT_BATCH_SIZE),
        params.get("offset", Config.DEFAULT_OFFSET),
    )

@task
def diagnose_and_heal(reviews: list[dict]) -> list[dict]:
    healed = [_heal_review(r) for r in reviews]
    logger.info(f"Healed {sum(1 for r in healed if
r['was_healed'])}/{len(reviews)}")
    return healed

@task
def analyze_sentiment(healed_reviews: list[dict],
                      model_info: dict) -> list[dict]:
    if not healed_reviews: return []
    return _analyze_with_ollama(healed_reviews, model_info)

@task
def aggregate_and_save(results: list[dict]) -> dict: ...

@task
def health_report(summary: dict) -> dict: ...

# Граф залежностей через передачу повернених значень
model_info = load_model()
reviews = load_reviews()
healed = diagnose_and_heal(reviews)
analyzed = analyze_sentiment(healed, model_info)
summary = aggregate_and_save(analyzed)
_ = health_report(summary)

business_analysis_pipeline_dag = business_analysis_pipeline()

```

Б.2 — batch_runner.py: послідовний та паралельний запуск

```

# airflow-pipeline/scripts/batch_runner.py

from concurrent.futures import ThreadPoolExecutor, as_completed
import subprocess, argparse, json, time

def trigger_dag_run(offset: int, batch_size: int,

```

```

        dry_run: bool = False) -> dict:
    conf = json.dumps({
        "batch_size": batch_size,
        "offset": offset,
    })
    cmd = ["airflow", "dags", "trigger",
           "self_healing_pipeline", "--conf", conf]

    if dry_run:
        print(f"[DRY RUN] offset={offset}, batch_size={batch_size}")
        return {"offset": offset, "status": "dry_run"}

    try:
        result = subprocess.run(cmd, capture_output=True, text=True,
                                timeout=30)
        if result.returncode == 0:
            print(f"Triggered: {offset:,} - {offset+batch_size:,}")
            return {"offset": offset, "status": "triggered"}
        return {"offset": offset, "status": "failed", "error":
result.stderr}
    except Exception as e:
        return {"offset": offset, "status": "error", "error": str(e)}

def run_batch_processing(total_records, batch_size,
                        parallel=1, start_offset=0,
                        delay=1.0, dry_run=False):
    offsets = list(range(start_offset, total_records, batch_size))
    print(f"Batches: {len(offsets)}, parallel={parallel}")

    if parallel == 1:
        for i, offset in enumerate(offsets):
            trigger_dag_run(offset, batch_size, dry_run)
            if not dry_run and i < len(offsets) - 1:
                time.sleep(delay)
    else:
        # ThreadPoolExecutor: N DAG-ів виконуються одночасно
        with ThreadPoolExecutor(max_workers=parallel) as executor:
            futures = {
                executor.submit(trigger_dag_run, offset, batch_size,
dry_run): offset
                for offset in offsets
            }
            for future in as_completed(futures):
                result = future.result()
                print(f"Done: {result}")

# Приклади використання:
# python batch_runner.py --total 5000000 --batch-size 1000
# python batch_runner.py --total 5000000 --batch-size 5000 --parallel 5
# python batch_runner.py --total 5000000 --batch-size 10000 --dry-run
# python batch_runner.py --total 5000000 --batch-size 1000 --start 100000
# відновлення

```

ДОДАТОК В

Реалізація Self-Healing та LLM-аналізу (Python)

В.1 — `_heal_review`: повна реалізація Chain of Responsibility

```
# airflow-pipeline/dags/agent_pipeline_dag.py

def _heal_review(review: dict) -> dict:
    text = review.get("text", "")

    # Базова структура результату — заповнюється для кожного запису
    result = {
        "review_id": review.get("review_id"),
        "business_id": review.get("business_id"),
        "stars": review.get("stars", 0),
        "original_text": None, # завжди зберігаємо оригінал
        "error_type": None,
        "action_taken": "none",
        "was_healed": False,
        "metadata": {
            "user_id": review.get("user_id"),
            "date": review.get("date"),
            "useful": review.get("useful", 0),
            "funny": review.get("funny", 0),
            "cool": review.get("cool", 0),
        }
    }

    # Зберігаємо оригінальний текст до будь-якого виправлення
    if isinstance(text, (str, int, float, bool, type(None))):
        result["original_text"] = text
    else:
        result["original_text"] = str(text) if text else None

    # — Chain of Responsibility —————
    # Умова 1: відсутній текст (None)
    if text is None:
        result["error_type"] = "missing_text"
        result["action_taken"] = "filled_with_placeholder"
        result["healed_text"] = "No review text provided."
        result["was_healed"] = True
        return result

    # Умова 2: хибний тип (не рядок)
    elif not isinstance(text, str):
        result["error_type"] = "wrong_type"
        try:
            converted = str(text).strip()
            result["healed_text"] = converted if converted else "No review text provided."
        except Exception:
            result["healed_text"] = "Conversion failed."
```

```

        result["action_taken"] = "type_conversion"
        result["was_healed"] = True

# Умова 3: порожній текст
elif not text.strip():
    result["error_type"] = "empty_text"
    result["healed_text"] = "No review text provided."
    result["action_taken"] = "filled_with_placeholder"
    result["was_healed"] = True

# Умова 4: тільки спецсимволи (відсутні алфавітно-цифрові)
elif not re.search(r"[a-zA-Z0-9]", text):
    result["error_type"] = "special_characters_only"
    result["healed_text"] = "[Non-text content]"
    result["action_taken"] = "replaced_special_characters"
    result["was_healed"] = True

# Умова 5: текст задовгий
elif len(text) > Config.MAX_TEXT_LENGTH:
    result["error_type"] = "too_long"
    result["healed_text"] = text[:Config.MAX_TEXT_LENGTH - 3] + "..."
    result["action_taken"] = "truncated_text"
    result["was_healed"] = True

# Нормальний шлях — текст валідний
else:
    result["healed_text"] = text.strip()
    result["was_healed"] = False

return result

```

B.2 — `_degraded_results`: Null Object патерн при недоступності Ollama

```

# Null Object Pattern — повертає нейтральні результати замість виключення
def _degraded_results(healed_reviews: list[dict],
                     error_message: str) -> list[dict]:
    """
    Викликається коли Ollama повністю недоступна.
    Забезпечує graceful degradation: конвеєр завершується успішно,
    а ступінь деградації фіксується у health-звіті.
    """
    return [
        {
            **review,
            "text": review.get("healed_text", ""),
            "predicted_sentiment": "NEUTRAL",
            "confidence": 0.5,
            "status": "degraded",
            "error_message": error_message,
        }
        for review in healed_reviews
    ]

```

```
]

```

B.3 — `_analyze_with_ollama`: retry-логіка та формування результатів

```
def _analyze_with_ollama(healed_reviews: list[dict],
                        model_info: dict) -> list[dict]:
    import ollama, time

    model_name = model_info.get("model_name")
    ollama_host = model_info.get("ollama_host", Config.OLLAMA_HOST)

    try:
        client = ollama.Client(host=ollama_host)
    except Exception as e:
        logger.error(f"Cannot connect to OLLAMA: {e}")
        return _degraded_results(healed_reviews, str(e)) # Null Object

    results = []
    total = len(healed_reviews)

    for idx, review in enumerate(healed_reviews):
        text = review.get("healed_text", "")
        prediction = None

        # Retry-цикл: 3 спроби з 1с затримкою між ними
        for attempt in range(Config.OLLAMA_RETRIES):
            try:
                prompt = f"""
                Analyze the sentiment of this review and classify it
                as POSITIVE, NEGATIVE, or NEUTRAL.
                Review: "{text}"
                Reply ONLY with JSON: {"sentiment": "POSITIVE",
"confidence": 0.95}}
                """
                response = client.chat(
                    model=model_name,
                    messages=[{"role": "user", "content": prompt}],
                    options={"temperature": 0.1}, # менше варіативності
                )
                prediction = _parse_ollama_response(
                    response["message"]["content"].strip()
                )
                break # успішна відповідь — виходимо з retry-циклу

            except Exception as e:
                if attempt < Config.OLLAMA_RETRIES - 1:
                    logger.warning(f"Attempt {attempt+1} failed: {e}.
Retrying...")
                    time.sleep(1)
                else:
                    logger.error(f"All retries failed for
{review['review_id']}: {e}")
```

```

        prediction = {"label": "NEUTRAL", "score": 0.5}

        # Логування прогресу кожні 10 записів
        # AirflowService.ParseAnalysisProgress() зчитує це для progress-
bar
        if (idx + 1) % 10 == 0 or (idx + 1) == total:
            logger.info(f"Analyzed {idx+1}/{total} reviews")

        results.append({
            "review_id":        review.get("review_id"),
            "business_id":      review.get("business_id"),
            "stars":            review.get("stars", 0),
            "text":              review.get("healed_text", ""),
            "original_text":    review.get("original_text", ""),
            "predicted_sentiment": prediction.get("label"),
            "confidence":       round(prediction.get("score"), 4),
            "status":            "healed" if review.get("was_healed")
else "success",
            "healing_applied":  review.get("was_healed"),
            "healing_action":   review.get("action_taken") if
review.get("was_healed") else None,
            "error_type":       review.get("error_type")   if
review.get("was_healed") else None,
            "metadata":         review.get("metadata", {}),
        })

    return results

```

B.4 — `_parse_ollama_response`: триступеневий парсинг відповіді

```

def _parse_ollama_response(response_text: str) -> dict:
    """
    Триступеневий парсинг відповіді llama3.2:
    1. Видалення markdown-обгортки ```json...```
    2. JSON-десеріалізація та валідація полів
    3. Fallback: пошук ключових слів POSITIVE/NEGATIVE у тексті
    """
    try:
        clean_text = response_text.strip()

        # Крок 1: видалення markdown ```json ... ```
        if clean_text.startswith("```"):
            lines = clean_text.split("\n")
            if lines[-1].strip() == "```":
                clean_text = "\n".join(lines[1:-1])
            else:
                clean_text = "\n".join(lines[1:])

        # Крок 2: JSON-парсинг
        parsed = json.loads(clean_text)
        sentiment = parsed.get("sentiment", "NEUTRAL").upper()
    
```

```

confidence = float(parsed.get("confidence", 0.0))

# Валідація допустимих значень sentiment
if sentiment not in ["POSITIVE", "NEGATIVE", "NEUTRAL"]:
    sentiment = "NEUTRAL"

# Нормалізація confidence до [0.0, 1.0]
return {
    "label": sentiment,
    "score": min(max(confidence, 0.0), 1.0)
}

except (json.JSONDecodeError, ValueError, KeyError, TypeError):
    # Крок 3: Fallback – пошук ключових слів у тексті
    upper_text = response_text.strip().upper()
    if "POSITIVE" in upper_text:
        return {"label": "POSITIVE", "score": 0.75}
    elif "NEGATIVE" in upper_text:
        return {"label": "NEGATIVE", "score": 0.75}
    # Нейтральний за замовчуванням
    return {"label": "NEUTRAL", "score": 0.5}

```

ДОДАТОК Г

Реалізація автентифікації та авторизації (C#)

Г.1 — AuthDbService: реєстрація, логін, валідація токену

```
// Dashboard/Services/AuthDbService.cs

public class AuthDbService
{
    private readonly string _connStr;

    public AuthDbService(IConfiguration config, IWebHostEnvironment env)
    {
        var dbPath = Path.Combine(env.ContentRootPath, "auth.db");
        _connStr = $"Data Source={dbPath}";
        EnsureSchema(); // ідемпотентне створення таблиць
    }

    private void EnsureSchema()
    {
        using var conn = Open();
        using var cmd1 = conn.CreateCommand();
        cmd1.CommandText = """
            CREATE TABLE IF NOT EXISTS users (
                id            INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT NOT NULL UNIQUE COLLATE NOCASE,
                hash          TEXT NOT NULL,
                salt          TEXT NOT NULL,
                created       TEXT NOT NULL
            )""";
        cmd1.ExecuteNonQuery();

        using var cmd2 = conn.CreateCommand();
        cmd2.CommandText = """
            CREATE TABLE IF NOT EXISTS tokens (
                token         TEXT PRIMARY KEY,
                user_id      INTEGER NOT NULL REFERENCES users(id) ON DELETE
CASCADE,
                username TEXT NOT NULL,
                created      TEXT NOT NULL,
                expires      TEXT NOT NULL
            )""";
        cmd2.ExecuteNonQuery();
    }

    // PBKDF2-SHA256, 100 000 ітерацій, 32-байтна сіль
    private static string Hash(string password, byte[] salt) =>
        Convert.ToBase64String(
            Rfc2898DeriveBytes.Pbkdf2(
                password, salt,
                iterations: 100_000,
                HashAlgorithmName.SHA256,
```

```

        outputLength: 32));

public async Task<(bool ok, string token, string error)>
    LoginAsync(string username, string password)
    {
        await using var conn = await OpenAsync();
        await using var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT id, hash, salt FROM users WHERE username
= $u COLLATE NOCASE";
        cmd.Parameters.AddWithValue("$u", username.Trim());

        await using var rdr = await cmd.ExecuteReaderAsync();
        if (!await rdr.ReadAsync())
            return (false, "", "Невірний логін або пароль");

        long    userId    = rdr.GetInt64(0);
        string storedHash = rdr.GetString(1);
        byte[] salt       = Convert.FromBase64String(rdr.GetString(2));
        await rdr.CloseAsync();

        if (Hash(password, salt) != storedHash)
            return (false, "", "Невірний логін або пароль");

        // Генерація токєну: 48 байт CSPRNG → Base64 (~384 біт ентропії)
        var rawToken =
Convert.ToBase64String(RandomNumberGenerator.GetBytes(48));
        var expires  = DateTime.UtcNow.AddDays(30);

        await using var ins = conn.CreateCommand();
        ins.CommandText = ""
            INSERT INTO tokens (token, user_id, username, created,
expires)
            VALUES ($t, $uid, $u, $c, $e)"";
        ins.Parameters.AddWithValue("$t", rawToken);
        ins.Parameters.AddWithValue("$uid", userId);
        ins.Parameters.AddWithValue("$u", username.Trim());
        ins.Parameters.AddWithValue("$c",
DateTime.UtcNow.ToString("o"));
        ins.Parameters.AddWithValue("$e", expires.ToString("o"));
        await ins.ExecuteNonQueryAsync();

        return (true, rawToken, "");
    }

public async Task<(bool valid, string username)>
ValidateTokenAsync(string token)
    {
        await using var conn = await OpenAsync();
        await using var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT username, expires FROM tokens WHERE
token = $t";
        cmd.Parameters.AddWithValue("$t", token);
        await using var rdr = await cmd.ExecuteReaderAsync();
        if (!await rdr.ReadAsync()) return (false, "");
    }

```

```

        string username = rdr.GetString(0);
        var expires = DateTime.Parse(rdr.GetString(1), null,
System.Globalization.DateTimeStyles.RoundtripKind);
        return expires > DateTime.UtcNow ? (true, username) : (false, "");
    }
}

```

Г.2 — TokenAuthHandler: валідація Bearer-токену у middleware

```

// Dashboard/Auth/TokenAuthHandler.cs

public class TokenAuthHandler(
    IOptionsMonitor<AuthenticationSchemeOptions> options,
    ILoggerFactory logger, UrlEncoder encoder,
    AuthDbService auth) // DI: отримуємо AuthDbService
    : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger,
encoder)
{
    protected override async Task<AuthenticateResult>
HandleAuthenticateAsync()
    {
        // Перевіряємо наявність заголовку Authorization
        if (!Request.Headers.TryGetValue("Authorization", out var header))
            return AuthenticateResult.NoResult();

        var value = header.ToString();
        if (!value.StartsWith("Bearer ",
StringComparison.OrdinalIgnoreCase))
            return AuthenticateResult.NoResult();

        // Витягуємо токен після "Bearer "
        var token = value["Bearer ".Length..].Trim();

        // Валідація через AuthDbService (перевірка у SQLite)
        var (valid, username) = await auth.ValidateTokenAsync(token);
        if (!valid)
            return AuthenticateResult.Fail("Invalid or expired token");

        // Формуємо ClaimsPrincipal — стає доступним як User у контролерах
        var claims = new[] { new Claim(ClaimTypes.Name, username) };
        var identity = new ClaimsIdentity(claims, Scheme.Name);
        var principal = new ClaimsPrincipal(identity);
        return AuthenticateResult.Success(
            new AuthenticationTicket(principal, Scheme.Name));
    }
}

```

Г.3 — AuthController: ендпоінти реєстрації, логіну, логoutu

```
// Dashboard/Controllers/AuthController.cs

[ApiController]
[Route("api/auth")]
public class AuthController(AuthDbService auth) : ControllerBase
{
    public record RegisterRequest(string Username, string Password);
    public record LoginRequest(string Username, string Password);

    [AllowAnonymous]
    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] RegisterRequest req)
    {
        var (ok, error) = await auth.RegisterAsync(req.Username, req.Password);
        if (!ok) return BadRequest(new { error });
        return Ok(new { message = "Реєстрацію успішно завершено" });
    }

    [AllowAnonymous]
    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest req)
    {
        var (ok, token, error) = await auth.LoginAsync(req.Username, req.Password);
        if (!ok) return Unauthorized(new { error });
        return Ok(new { token, username = req.Username.Trim() });
    }

    [Authorize]
    [HttpPost("logout")]
    public async Task<IActionResult> Logout()
    {
        // Витягуємо токен з заголовку та видаляємо з БД
        var rawHeader = Request.Headers["Authorization"].ToString();
        var token = rawHeader.Substring("Bearer ".Length).Trim();
        await auth.RevokeTokenAsync(token);
        return Ok(new { message = "Вихід виконано" });
    }

    [Authorize]
    [HttpGet("me")]
    public IActionResult Me()
    {
        // User.FindFirstValue — значення з ClaimsPrincipal, сформованого TokenAuthHandler
        var username = User.FindFirstValue(ClaimTypes.Name) ?? "";
        return Ok(new { username });
    }
}
```

Г.4 — AuthStateService (WASM) та MainLayout: клієнтська auth

```
// Dashboard.Client/Services/AuthStateService.cs

public class AuthStateService(HttpClient http, IJSRuntime js)
{
    private const string TokenKey = "auth_token";

    public bool    IsAuthenticated { get; private set; }
    public string  Username        { get; private set; } = "";
    public bool    Initialized     { get; private set; }
    public event  Action? StateChanged;

    // Викликається з MainLayout при завантаженні застосунку
    public async Task InitializeAsync()
    {
        if (Initialized) return;
        try
        {
            // Зчитуємо токен з localStorage через JS Interop
            var token = await
js.InvokeAsync<string?>("localStorage.getItem", TokenKey);
            if (!string.IsNullOrEmpty(token))
            {
                SetAuthHeader(token);
                var resp = await http.GetAsync("api/auth/me");
                if (resp.IsSuccessStatusCode)
                {
                    var data = await
resp.Content.ReadFromJsonAsync<MeResponse>();
                    IsAuthenticated = true;
                    Username        = data?.Username ?? "";
                }
                else
                    await ClearTokenAsync(); // токен прострочений
            }
        }
        catch { /* JS недоступний під час pre-render */ }
        finally { Initialized = true; StateChanged?.Invoke(); }
    }

    public async Task<(bool ok, string error)> LoginAsync(string username,
string password)
    {
        var resp = await http.PostAsJsonAsync("api/auth/login", new {
username, password });
        if (!resp.IsSuccessStatusCode)
        {
            var err = await
resp.Content.ReadFromJsonAsync<ErrorResponse>();
            return (false, err?.Error ?? "Помилка входу");
        }
    }
}
```

```

        var data = await resp.Content.ReadFromJsonAsync<LoginResponse>();
        if (data?.Token is null) return (false, "Помилка входу");

        await js.InvokeVoidAsync("localStorage.setItem", TokenKey,
data.Token);
        SetAuthHeader(data.Token);
        IsAuthenticated = true;
        Username = data.Username ?? username.Trim();
        StateChanged?.Invoke();
        return (true, "");
    }

    // Встановлює Bearer-заголовок для всіх подальших запитів
    private void SetAuthHeader(string token) =>
        http.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);
    }

    // Dashboard.Client/Layout/MainLayout.razor – захист маршрутів
    @code {
        protected override async Task OnInitializedAsync()
        {
            Auth.StateChanged += OnAuthChanged;
            await Auth.InitializeAsync();
            RedirectIfNeeded();
        }

        private void RedirectIfNeeded()
        {
            var uri = Navigation.Uri;
            // Незаавторизований → /login
            if (Auth.Initialized && !Auth.IsAuthenticated &&
!uri.Contains("/login"))
                Navigation.NavigateTo("/login", replace: true);
            // Авторизований на /login → /
            else if (Auth.Initialized && Auth.IsAuthenticated &&
uri.Contains("/login"))
                Navigation.NavigateTo("/", replace: true);
        }
    }
}

```

ДОДАТОК Д

AirflowService, Polling, Program.cs, SVG (C#)

Д.1 — Program.cs: реєстрація сервісів та middleware

```
// Dashboard/Program.cs

var builder = WebApplication.CreateBuilder(args);

// Blazor WASM Hosted: рендер компонентів на сервері + WASM на клієнті
builder.Services.AddRazorComponents()
    .AddInteractiveWebAssemblyComponents();
builder.Services.AddControllers();
builder.Services.AddHttpClient();

// Сервіси DI
builder.Services.AddSingleton<YelpImportService>(); // Singleton:
// зберігає стан імпорту
builder.Services.AddScoped<BusinessService>(); // Scoped: новий
// екземпляр на запит
builder.Services.AddScoped<AirflowService>();
builder.Services.AddScoped<AnalyticsService>();
builder.Services.AddScoped<AuthDbService>();

// Власна схема автентифікації: Bearer-токени через SQLite
builder.Services.AddAuthentication("Bearer")
    .AddScheme<AuthenticationSchemeOptions, TokenAuthHandler>("Bearer", _
=> { });
builder.Services.AddAuthorization();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();
app.UseAntiforgery();

app.MapControllers();
app.MapRazorComponents<App>()
    .AddInteractiveWebAssemblyRenderMode()
    .AddAdditionalAssemblies(typeof(Dashboard.Client._Imports).Assembly);

app.Run();
```

Д.2 — AirflowService: повна реалізація з кешуванням токєну

```
// Dashboard/Services/AirflowService.cs (ключові методи)

// Статичне кешування токєну Airflow (SemaphoreSlim для потокобезпеки)
private static string? _token;
private static DateTime _tokenExpiry = DateTime.MinValue;
private static readonly SemaphoreSlim _tokenLock = new(1, 1);

private async Task<string> GetTokenAsync(CancellationTokєn ct)
{
    if (_token != null && DateTime.UtcNow < _tokenExpiry) return _token;

    await _tokenLock.WaitAsync(ct);
    try
    {
        if (_token != null && DateTime.UtcNow < _tokenExpiry) return
_token;

        // Читаємо пароль з файлу
        simple_auth_manager_passwords.json.generated
        var password = ReadPassword();
        var client = _factory.CreateClient();
        var body = JsonSerializer.Serialize(new { username = "admin",
password });
        var resp = await client.PostAsync($"{Base}/auth/token",
new StringContent(body, Encoding.UTF8,
"application/json"), ct);

        if (!resp.IsSuccessStatusCode)
            throw new Exception($"Airflow auth failed:
{(int)resp.StatusCode}");

        using var doc = JsonDocument.Parse(await
resp.Content.ReadAsStringAsync(ct));
        _token =
doc.RootElement.GetProperty("access_token").GetString();
        _tokenExpiry = DateTime.UtcNow.AddMinutes(55); // токєн Airflow
живє ~60 хв
        return _token;
    }
    finally { _tokenLock.Release(); }
}

// Запуск DAG через REST API v2
public async Task<(string runId, DateTime startTime)> TriggerDagAsync(
    string dagId, object conf, CancellationTokєn ct = default)
{
    var client = await AuthClientAsync(ct);
    var body = JsonSerializer.Serialize(new {
        logical_date = DateTime.UtcNow.ToString("o"), conf
    });
    var resp = await client.PostAsync(
```

```

        $"{Base}/api/v2/dags/{dagId}/dagRuns",
        new StringContent(body, Encoding.UTF8, "application/json"), ct);

    if (!resp.IsSuccessStatusCode)
        throw new Exception($"Airflow {(int)resp.StatusCode}");

    using var doc = JsonDocument.Parse(await
resp.Content.ReadAsStringAsync(ct));
    var root = doc.RootElement;
    var runId = (root.TryGetProperty("dag_run_id", out var r1) ?
r1.GetString() : null)
        ?? root.GetProperty("run_id").GetString()!;
    return (runId, DateTime.UtcNow);
}

// Парсинг прогресу аналізу з логів Airflow
public (int current, int total) ParseAnalysisProgress(string logs)
{
    // Шукаємо рядки виду: "Analyzed 50/100 reviews"
    var matches = Regex.Matches(logs, @"Analyzed (\d+)/(\d+) reviews");
    if (matches.Count == 0) return (0, 0);
    var last = matches[1];
    return (int.Parse(last.Groups[1].Value),
int.Parse(last.Groups[2].Value));
}

// Скасування DAG run через PATCH зі станом "failed"
public async Task CancelDagRunAsync(string dagId, string runId,
CancellationToken ct = default)
{
    var client = await AuthClientAsync(ct);
    var body = JsonSerializer.Serialize(new { state = "failed" });
    var req = new HttpRequestMessage(HttpMethod.Patch,
        $"{Base}/api/v2/dags/{dagId}/dagRuns/{runId}")
    { Content = new StringContent(body, Encoding.UTF8, "application/json")
    };
    await client.SendAsync(req, ct);
}

```

Д.3 — StartPollingAsync та PollOnceAsync (BusinessAnalysis.razor)

```

// Dashboard.Client/Pages/BusinessAnalysis.razor — polling DAG прогресу

private async Task StartPollingAsync()
{
    _pollCts?.Cancel();
    _pollCts = new CancellationTokenSource();
    var timer = new PeriodicTimer(TimeSpan.FromSeconds(5));
    try
    {
        while (await timer.WaitForNextTickAsync(_pollCts.Token))
    }
}

```

```

        {
            await PollOnceAsync(_pollCts.Token);
            await InvokeAsync(StateHasChanged); // оновити Blazor UI

            if (_dagDone || _dagFailed || _userCancelled)
                break;
        }
    }
    catch (OperationCanceledException) { }
    finally { timer.Dispose(); }
}

private async Task PollOnceAsync(Cancellation_token ct)
{
    if (_dagRunId == null) return;
    try
    {
        // 1. Стартує DAG run
        var runResp = await Http.GetFromJsonAsync<DagRunDto>(
            $"api/airflow/dag-run/{Uri.EscapeDataString(_dagRunId)}", ct);

        if (runResp != null)
        {
            _dagState = runResp.State;
            if (runResp.State == "success")
            {
                _dagRunning = false; _dagDone = true;
                // Обчислюємо тривалість виконання
                if (DateTime.TryParse(runResp.StartDate, out var s) &&
                    DateTime.TryParse(runResp.EndDate, out var e))
                    _dagDuration = (e - s).ToString(@"hh\:mm\:ss");
                // Шукаємо output-файл результату
                _outputFile = await Http.GetFromJsonAsync<OutputFileDto>(
                    $"api/airflow/output-
file/{Uri.EscapeDataString(_selected!.BusinessId)}", ct);
            }
            else if (runResp.State == "failed")
            {
                _dagRunning = false; _dagFailed = true; }
        }

        // 2. Стани тасків (таймлайн у UI)
        var tasks = await Http.GetFromJsonAsync<List<TaskInstanceDto>>(
            $"api/airflow/task-
instances/{Uri.EscapeDataString(_dagRunId)}", ct);
        if (tasks != null) _taskInstances = tasks;

        // 3. Логів analyze_sentiment → прогрес аналізу відгуків
        var logs = await Http.GetFromJsonAsync<TaskLogsDto>(
            $"api/airflow/task-
logs/{Uri.EscapeDataString(_dagRunId)}/analyze_sentiment", ct);
        if (logs != null)
        {
            if (logs.Total > 0) { _progCurrent = logs.Current; _progTotal
= logs.Total; }
            if (logs.LastLines.Count > 0) _logLines = logs.LastLines;
        }
    }
    catch { }
}

```

```

    }
}
catch (OperationCanceledException) { throw; }
catch { /* ігноруємо тимчасові мережеві помилки між тиками */ }
}

// IAsyncDisposable – зупиняємо таймер при навігації геть
public async ValueTask DisposeAsync()
{
    _pollCts?.Cancel();
    _pollCts?.Dispose();
    await Task.CompletedTask;
}

```

Д.4 — SVG-генератори: Heatmap та DrawHealingBarSvg

```

// BusinessAnalysis.razor – генерація SVG безпосередньо у С# (спрощено)

// Heatmap 7×24: день тижня × година доби
private MarkupString DrawHeatmapSvg()
{
    const int ROWS = 7, COLS = 24, CELL = 24, GAP = 2, LEFT = 34, TOP = 30;

    int W = LEFT + COLS * (CELL + GAP) + 10;
    int H = TOP + ROWS * (CELL + GAP) + 30;

    // Заповнюємо сітку даними DuckDB (день, година, кількість)
    int[,] grid = new int[ROWS, COLS];
    foreach (var c in _heatmapData)
    {
        int row = c.DayOfWeek == 0 ? 6 : c.DayOfWeek - 1;
        if (row >= 0 && row < ROWS && c.Hour >= 0 && c.Hour < COLS)
            grid[row, c.Hour] = c.Count;
    }

    int maxCount = 0;
    for (int r = 0; r < ROWS; r++)
        for (int c = 0; c < COLS; c++)
            if (grid[r, c] > maxCount) maxCount = grid[r, c];
    if (maxCount == 0) maxCount = 1;

    var sb = new StringBuilder();
    sb.Append($"<svg viewBox=\"0 0 {W} {H}\" style=\"width:100%\">");

    string[] days = { "Пн", "Вт", "Ср", "Чт", "Пт", "Сб", "Нд" };
    for (int r = 0; r < ROWS; r++)
        for (int c = 0; c < COLS; c++)
        {
            double cx = LEFT + c * (CELL + GAP);
            double cy = TOP + r * (CELL + GAP);
            int count = grid[r, c];

```

```

        // Лінійна інтерполяція кольору: темно-сірий → яскраво-зелений
        double t = (double)count / maxCount;
        string fill = count == 0 ? "#1f2937"
            : $"rgb(({int}(17 + t*57)),{int}(24 + t*198)),{int}(39 +
t*89))");
        sb.Append($"<rect x=\"{cx:F0}\" y=\"{cy:F0}\" width=\"{CELL}\"
            height=\"{CELL}\" fill=\"{fill}\" rx=\"3\"/>");
    }
    sb.Append("</svg>");
    return (MarkupString) sb.ToString();
}

// Горизонтальний bar chart для healing actions
private MarkupString DrawHealingBarSvg()
{
    var stats = _summary?.HealingStats;
    if (stats == null || stats.Count == 0)
        return (MarkupString)"<div>Hemae healing actions</div>";

    var items = stats.OrderByDescending(kv => kv.Value).ToList();
    const int W = 480, ml = 160, barH = 26, gap = 8;
    int maxVal = items.Max(kv => kv.Value);
    int barW = W - ml - 55;
    int H = 10 + items.Count * (barH + gap) + 20;

    var sb = new StringBuilder();
    sb.Append($"<svg viewBox=\"0 0 {W} {H}\" style=\"width:100%\">");
    for (int i = 0; i < items.Count; i++)
    {
        int y = 10 + i * (barH + gap);
        double w = (double)items[i].Value * barW / maxVal;
        sb.Append($"<text x=\"{ml-8}\" y=\"{y+barH/2+4}\"
            text-anchor=\"end\"
fill=\"<div>Hemae healing actions</div>";
        if (w > 0)
            sb.Append($"<rect x=\"{ml}\" y=\"{y}\" width=\"{w:F1}\"
                height=\"{barH}\" fill=\"<div>Hemae healing actions</div>";
        rx="4"/>");
        sb.Append($"<text x=\"{ml+w+6:F1}\" y=\"{y+barH/2+4}\"
            fill=\"<div>Hemae healing actions</div>";
    }
    sb.Append("</svg>");
    return (MarkupString) sb.ToString();
}

```

ДОДАТОК Е

Конфігурація розгортання

Е.1 — docker-compose.yml (Airflow)

```
# airflow-pipeline/docker-compose.yml

version: "3.8"
services:
  airflow:
    build: .
    command: standalone # всі компоненти Airflow в одному процесі
    user: "0:0"
    environment:
      - AIRFLOW_HOME=/opt/airflow
      - AIRFLOW_CORE_LOAD_EXAMPLES=False
      -
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=sqlite:///opt/airflow/airflow.db
      - AIRFLOW__CORE__DAGS_FOLDER=/opt/airflow/dags
      - PIPELINE_BASE_DIR=/opt/airflow
      -
    PIPELINE_INPUT_FILE=/opt/airflow/input/yelp_academic_dataset_review.json
      - PIPELINE_OUTPUT_DIR=/opt/airflow/output
      - OLLAMA_HOST=http://host.docker.internal:11434 # Ollama на хост-
машині
      - OLLAMA_MODEL=llama3.2
      - OLLAMA_TIMEOUT=120
      - OLLAMA_RETRIES=3
    ports:
      - "8080:8080"
    volumes:
      - ../opt/airflow # монтуємо всю директорію — DAG-и змінюються без
перебудови
    extra_hosts:
      - "host.docker.internal:host-gateway" # дозволяє контейнеру
звертатись до хоста
```

Е.2 — appsettings.json (Blazor Dashboard)

```
// blazor-dashboard/Dashboard/appsettings.json
{
  // Відносний шлях від ContentRootPath до директорії output Airflow
  "PipelineOutputPath": "../..../airflow-pipeline/output",
  // Базовий шлях до Airflow-pipeline (для DuckDB, input, output)
  "PipelineBasePath": "../..../airflow-pipeline",
  // URL Airflow UI та REST API
  "AirflowUrl": "http://localhost:8080/",
  "Logging": {
    "LogLevel": {
```

```

        "Default": "Information",
        "Microsoft.AspNetCore": "Warning"
    }
},
"AllowedHosts": "*"
}

// Зчитування шляхів у сервісах (приклад з AnalyticsService):
private string DbPath => Path.GetFullPath(Path.Combine(
    _env.ContentRootPath,
    _config["PipelineBasePath"] ?? "../..../airflow-pipeline",
    "input", "yelp.duckdb"));

```

E.3 — requirements.txt (Python-залежності Airflow)

```

# airflow-pipeline/requirements.txt

# apache-airflow>=3.0.6    # встановлюється окремо
apache-airflow-providers-fab>=3.0.0    # FlaskAppBuilder auth для Airflow
UI

# ML / NLP
ollama>=0.6.0              # Python-клієнт для Ollama REST API
transformers                # для можливих майбутніх fine-tuned моделей
torch                      # бекенд для transformers

psycopg2-binary>=2.9.0     # PostgreSQL (резерв для production Airflow)
pytest                     # unit-тести модулів DAG

```